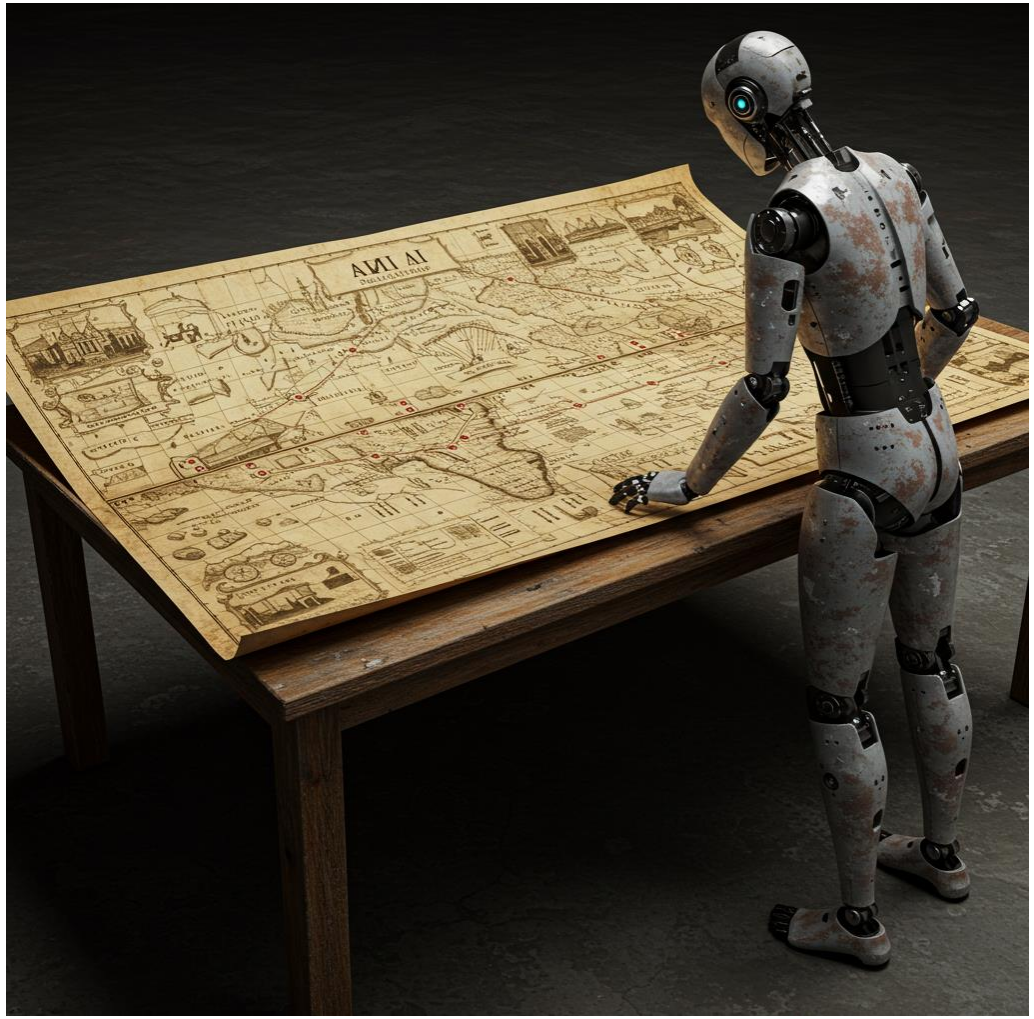
Agentic AI in Software Engineering



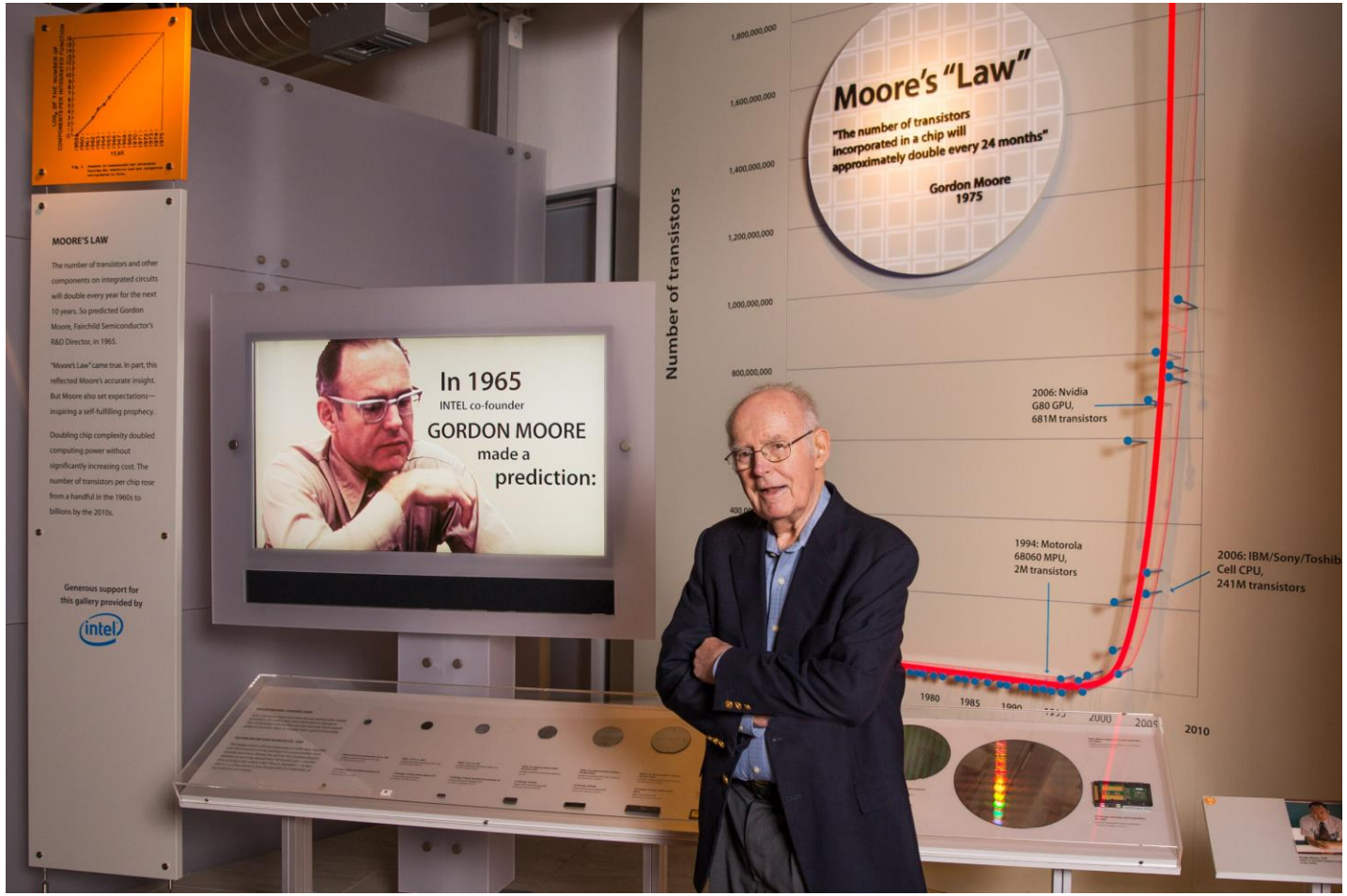
A TIMELINE

April 5th, 2025 Vincent Hellendoorn Google Deepmind & CMU

Timelines of exponential developments

Classical Computing:

Transistors on a chip



Computer History Museum – https://computerhistory.org

past present future

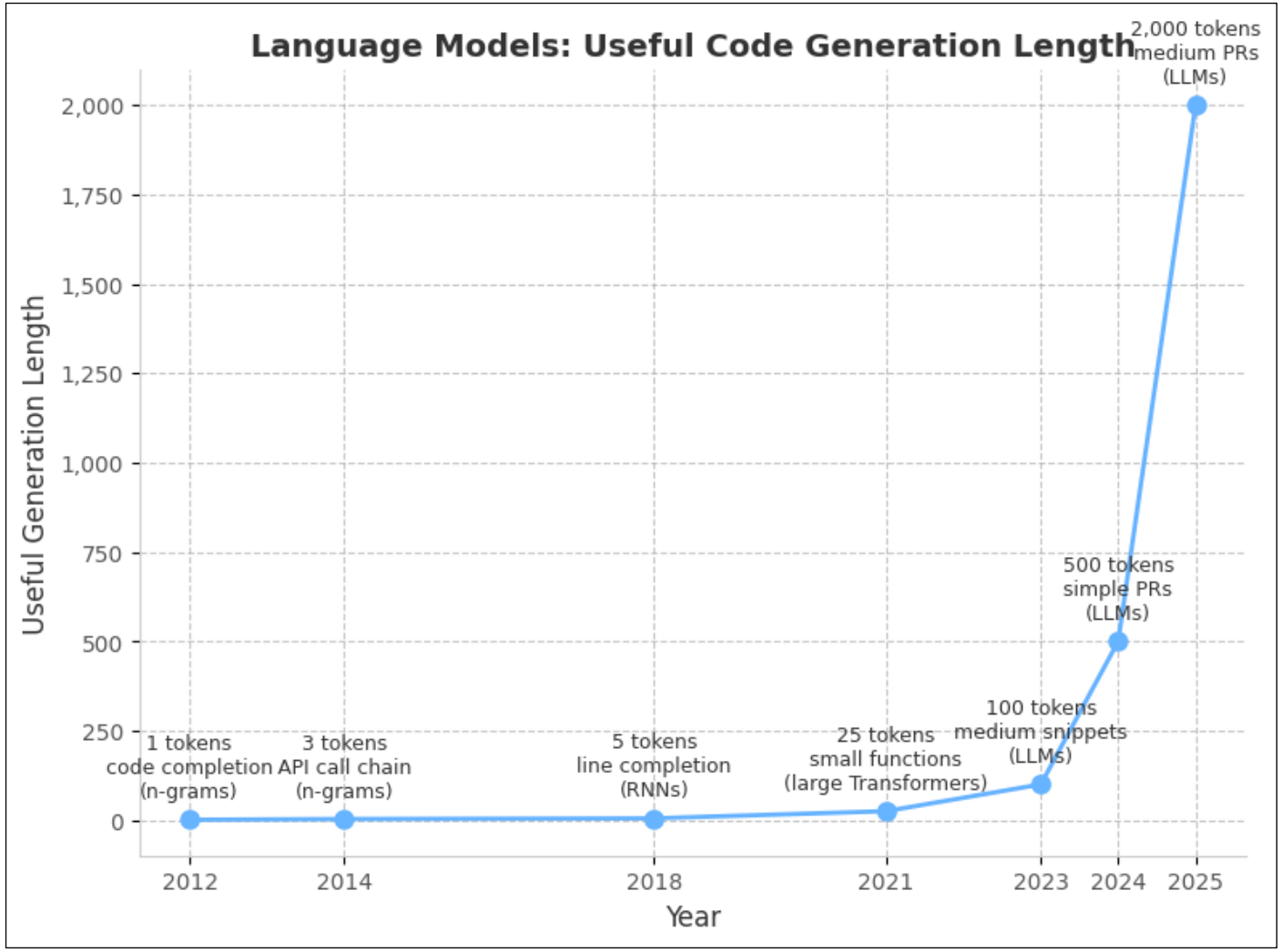
Timelines of exponential developments

Classical Computing:

Transistors on a chip

Language Models:

Useful generation length



https://paperswithcode.com/sota/language-modelling-on-wikitext-103

Guiding Principle

Information out ≤ information in

This Talk

The past: basics & evolution of code language models

This Talk

The past: basics & evolution of code language models

Today: in-depth on coding agents

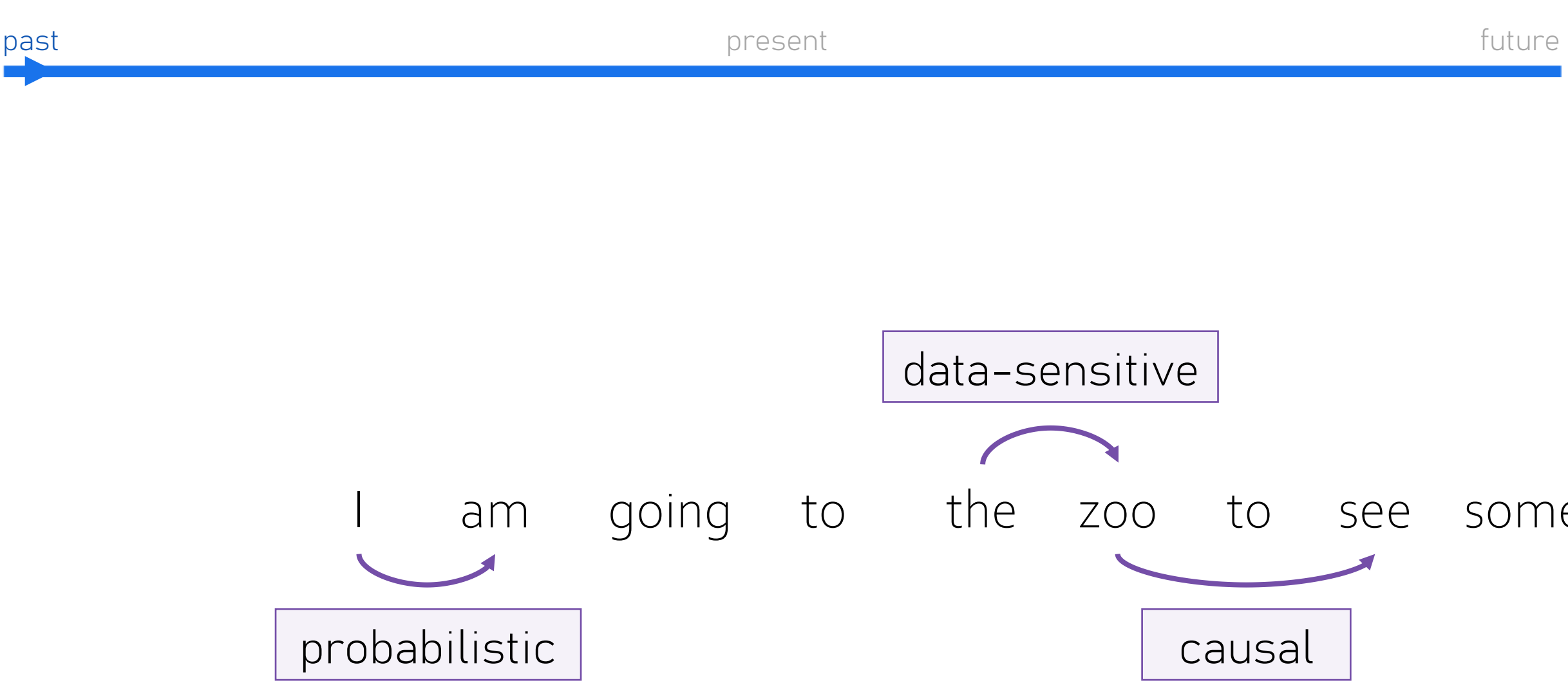
This Talk

The past: basics & evolution of code language models

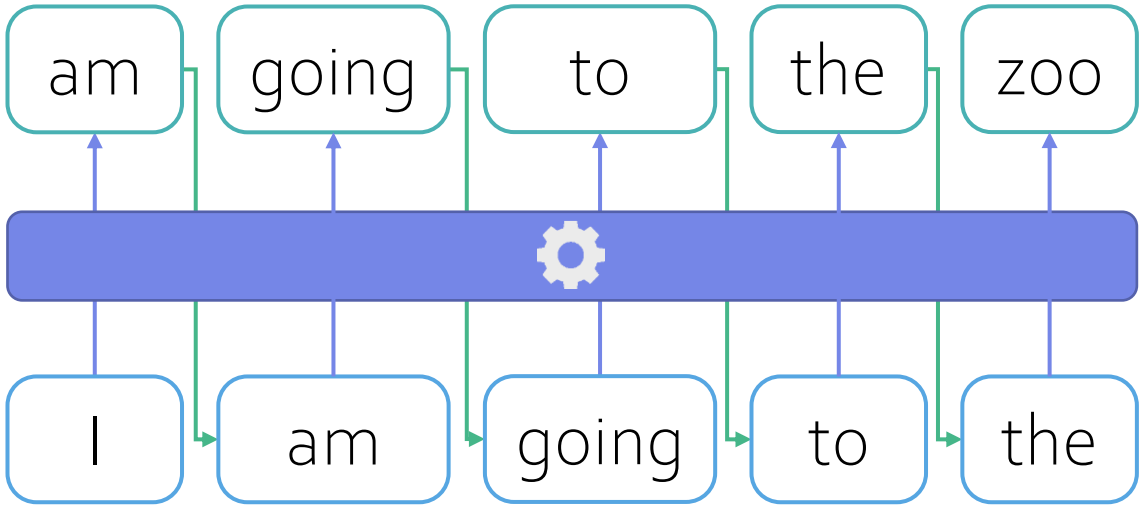
Today: in-depth on coding agents

Next: the changing role of software engineers

I am going to the zoo to see some


past
present
future
data-sensitive
I am going to the zoo to see some
probabilistic
causal

past present future



What model?

I am going to the
- movies
- grocery store
- meeting
- zoo
- ???

n-gram models

	movies	*seen **10** times*
	grocery store	*seen **50** times*
I am going to the	meeting	*seen **12** times*
	zoo	*seen **40** times*
	All else	*seen **88** times*

n-gram models

	movies	*seen **10** times*	*5%*
	grocery store	*seen **50** times*	*25%*
I am going to the	meeting	*seen **12** times*	*6%*
	zoo	*seen **40** times*	*20%*
	All else	*seen **88** times*	*44%*

n-gram models

I am going to th

movies 5%

seen 10 times 25%

6%

20%

All else seen 88 times 44%

Problems?

Language models are all:

Context-sensitive

Approximate

Distribution-specific

Did you see the new elephants?

Did you see the new elephants?

I am going to the ___

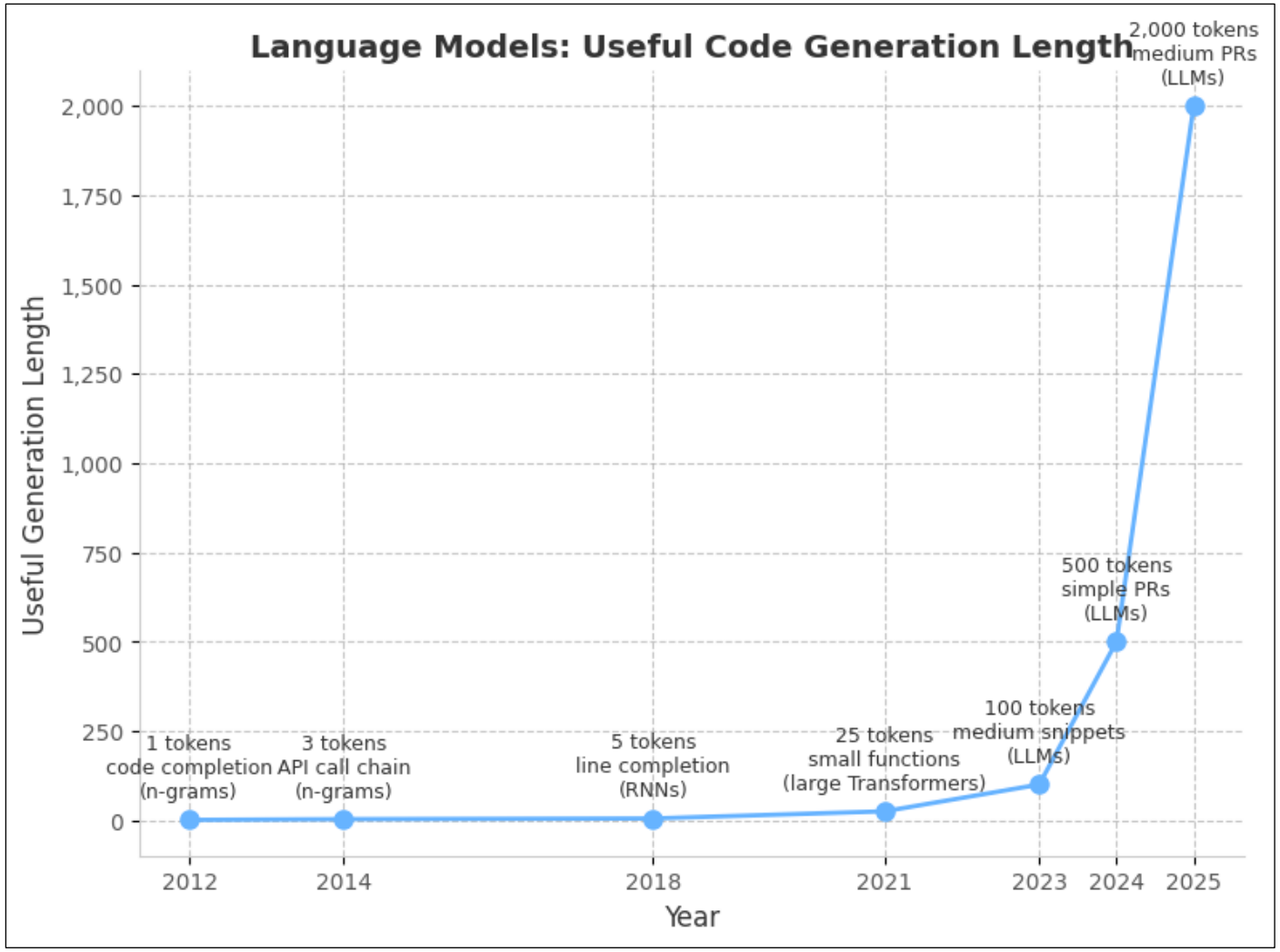
I am going to the ___

So how did we get here?

Larger, better models

Better data curation

Better use of context

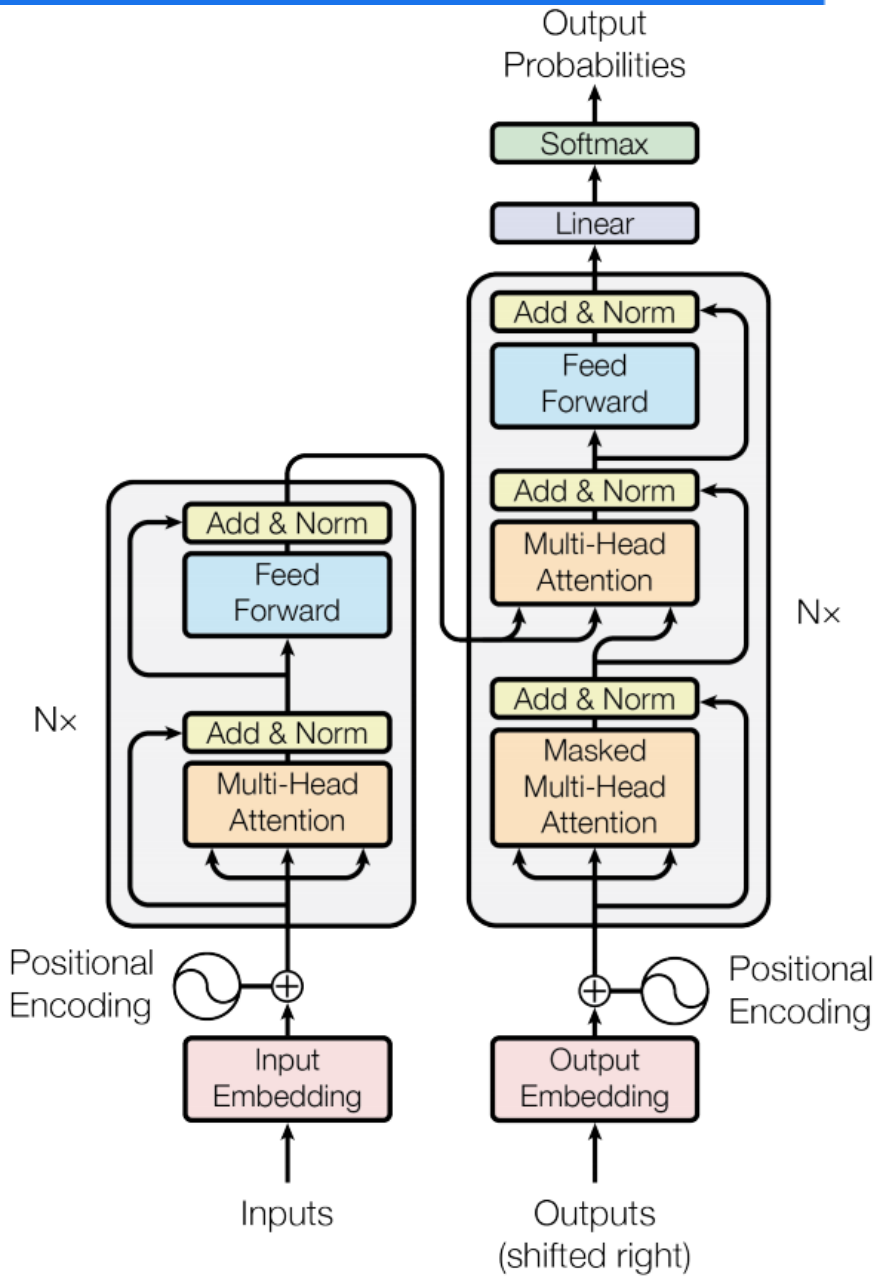


Neural Language Models

Are complicated

I won't tell you how they work

It's not a secret, it just takes a long time

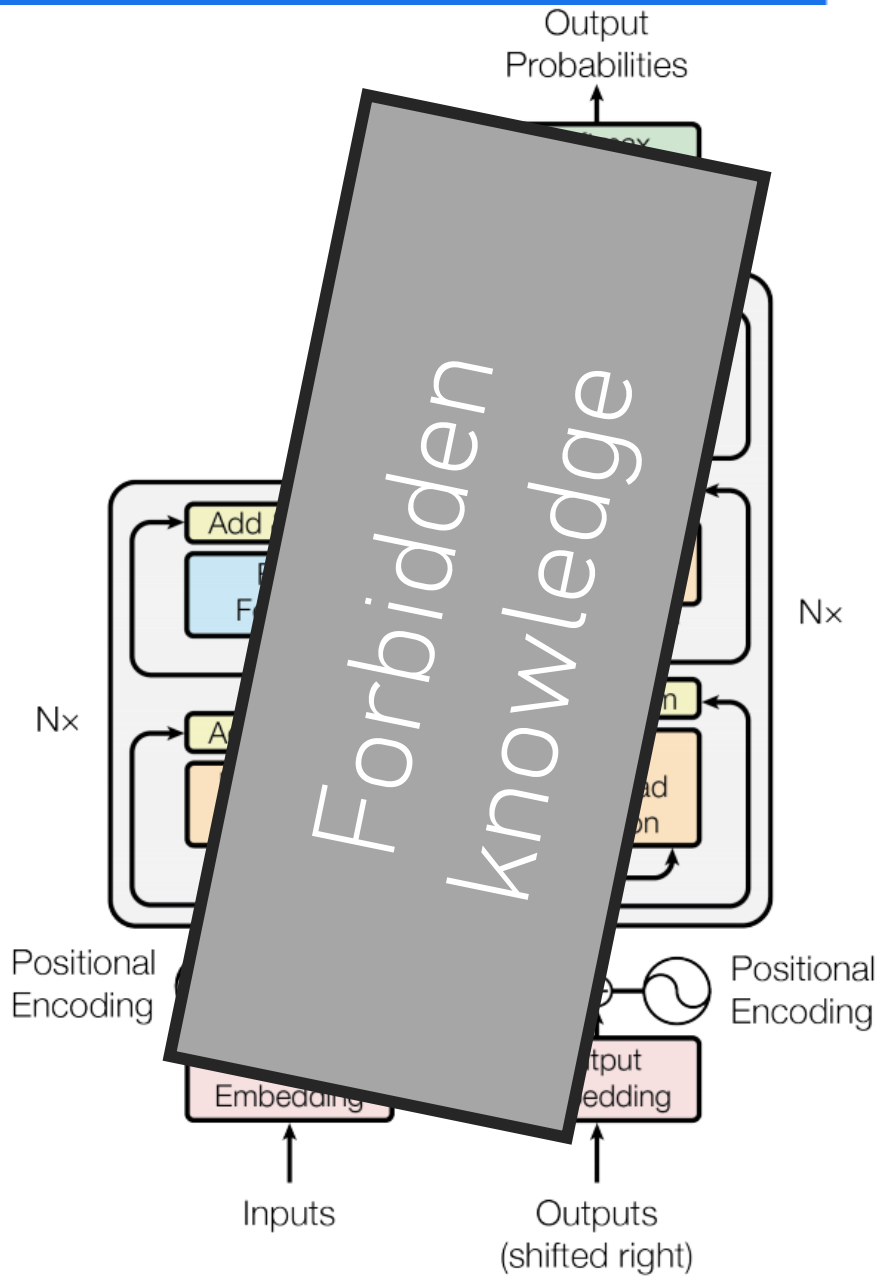


Attention is All You Need. Vaswani et al., 2017

Neural Language Models

Cliff's notes:

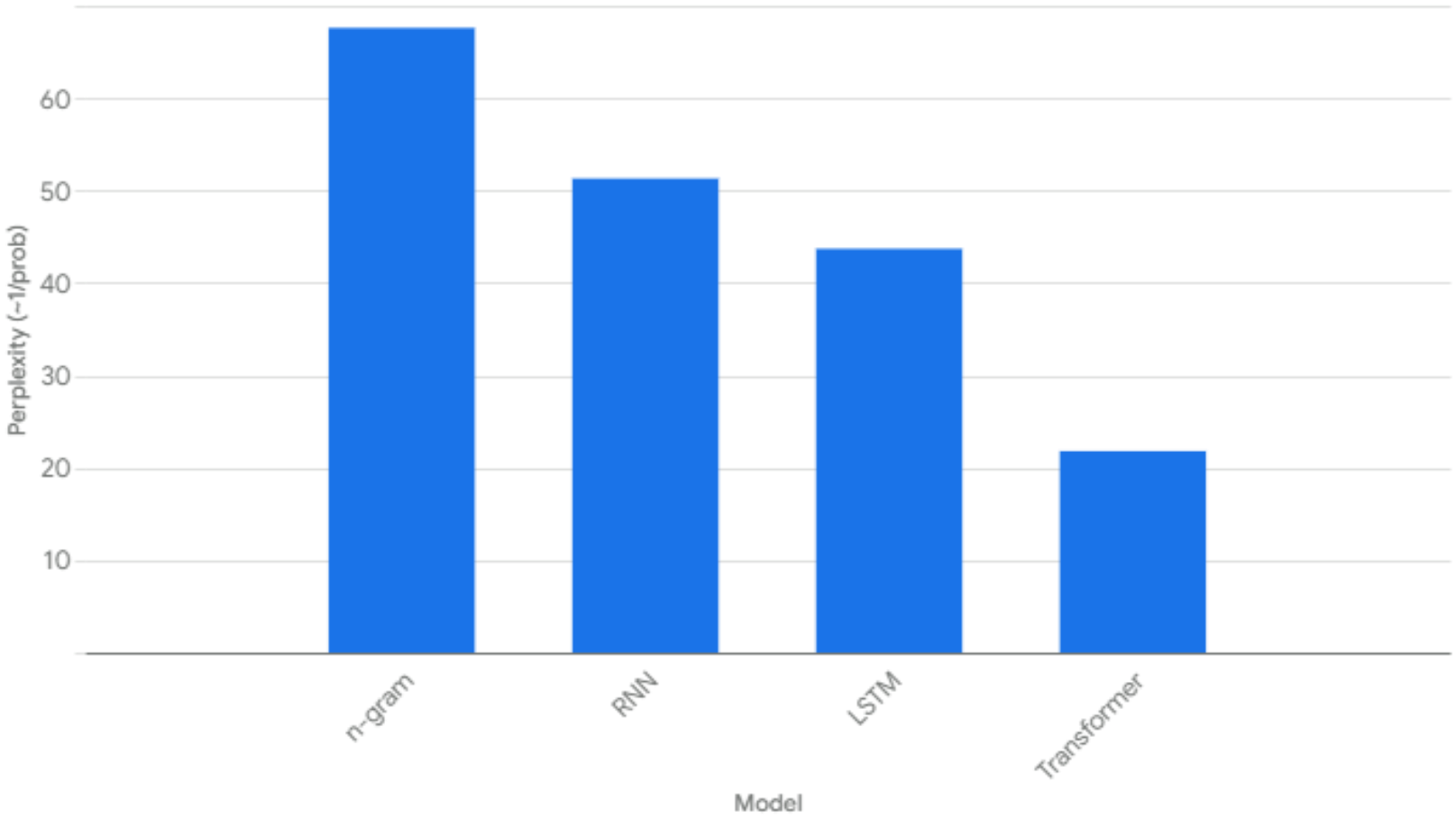
- Context → high-dimensional representation
- Representation → next word probability distribution
- Difference with real word → loss
- Loss + back-propagation → better representation
- Scale, scale, scale



Attention is All You Need. Vaswani et al., 2017

A timeline of language models

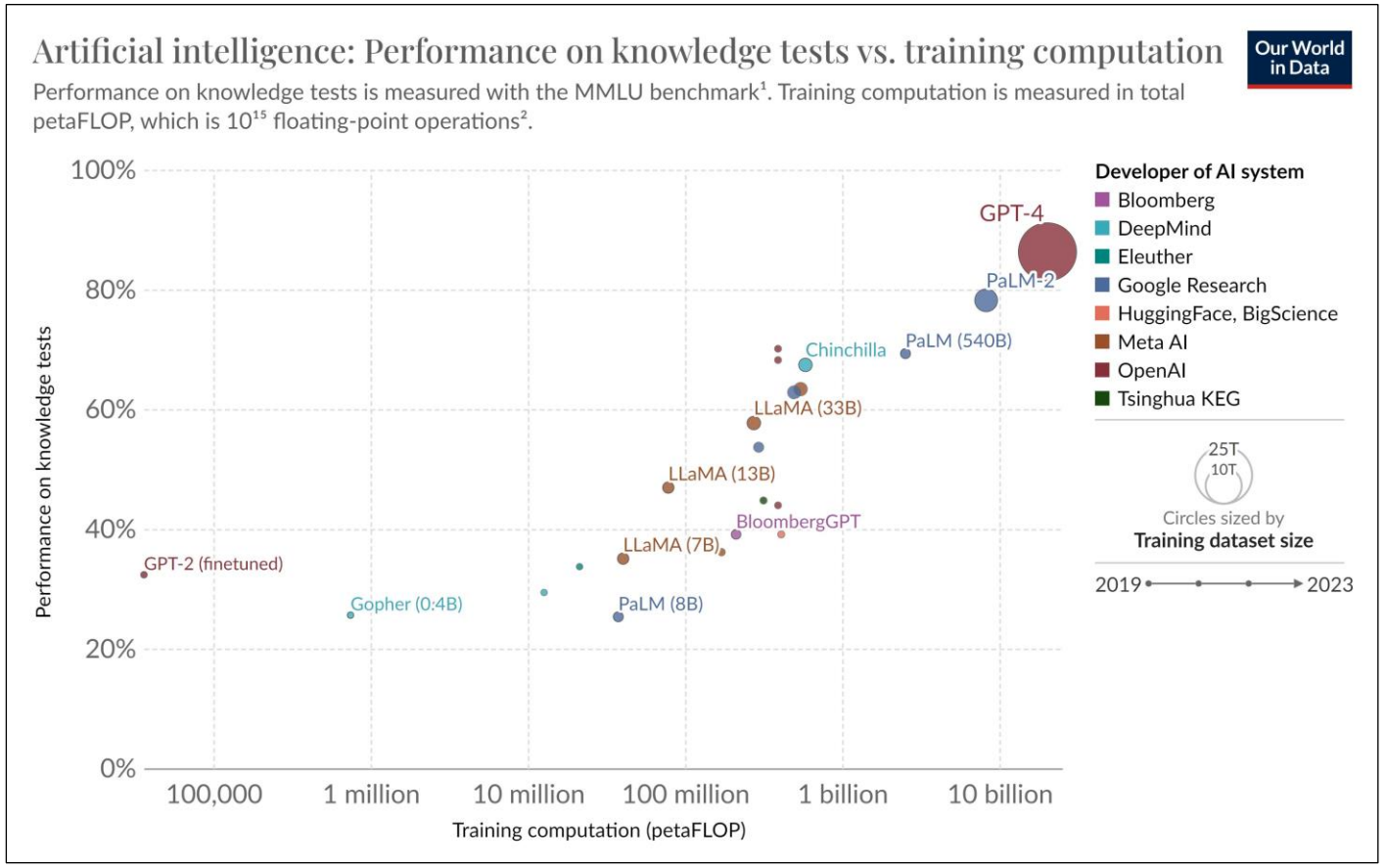
Pre-2020
(lower is better)



One Billion Word Language Modeling Benchmark – select models

A timeline of language models

2019 – 2023



Published online at OurWorldInData.org. Retrieved from: 'https://ourworldindata.org/artificial-intelligence'

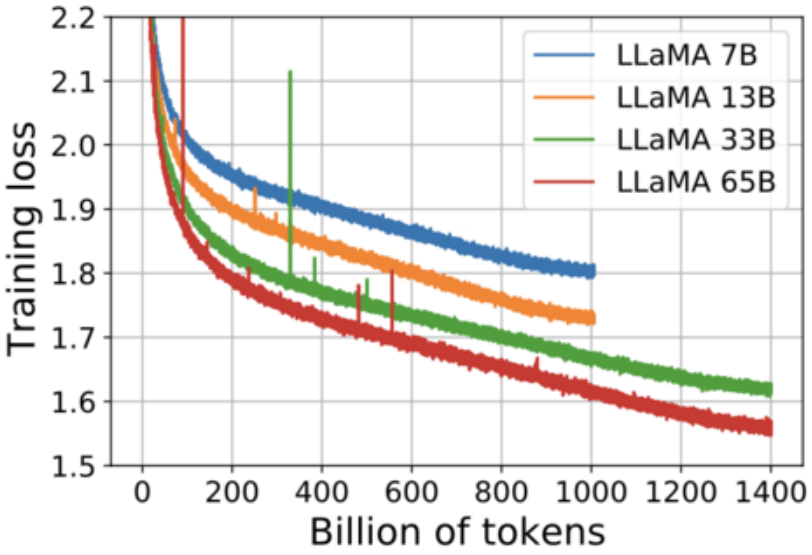
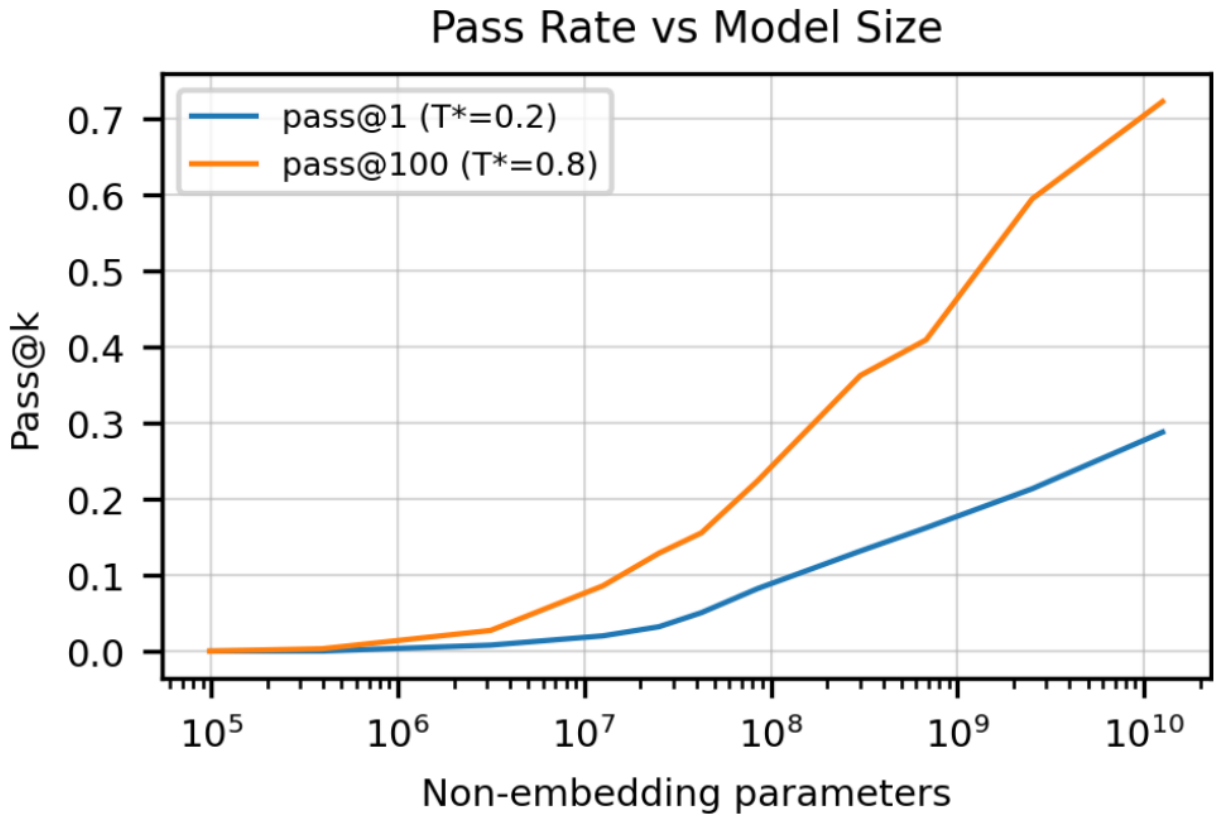
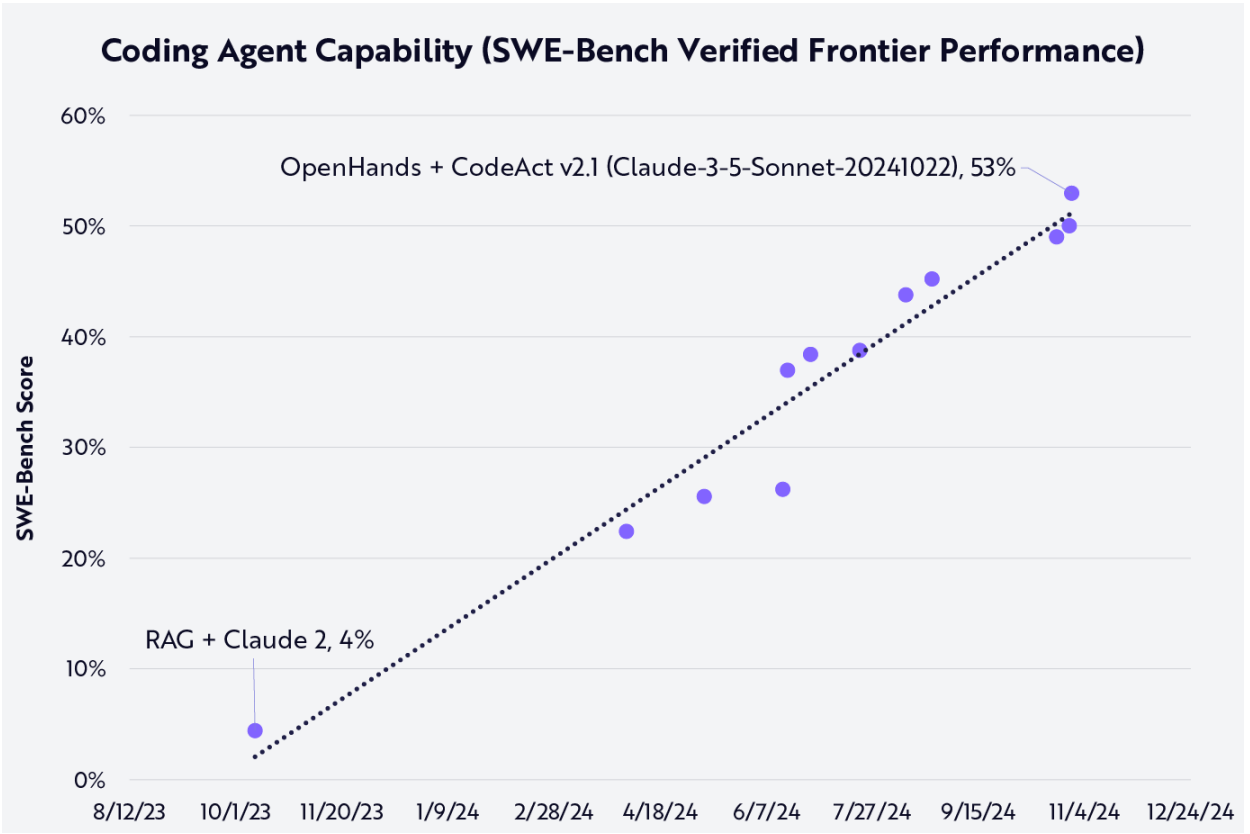


Figure 1: **Training loss over train tokens for the 7B, 13B, 33B, and 65 models.** LLaMA-33B and LLaMA-65B were trained on 1.4T tokens. The smaller models were trained on 1.0T tokens. All models are trained with a batch size of 4M tokens.

Llama report, https://arxiv.org/abs/2302.13971

How about code language models?





https://arxiv.org/abs/2107.03374

ARK Investment Management LLC, 2024

Why don't you update this app & develop a GUI for it! #52

Closed as not planned

PCUser2021 opened on May 6, 2021

Why don't you update this app & develop a GUI for it!

2 70 57 2 4 5

mbonne on May 6, 2021 via email

Why don't you PCUser2021!!

...

4 149 2 3

PCUser2021 on May 7, 2021 Author

What do you mean?

1 30 24 2 7 4

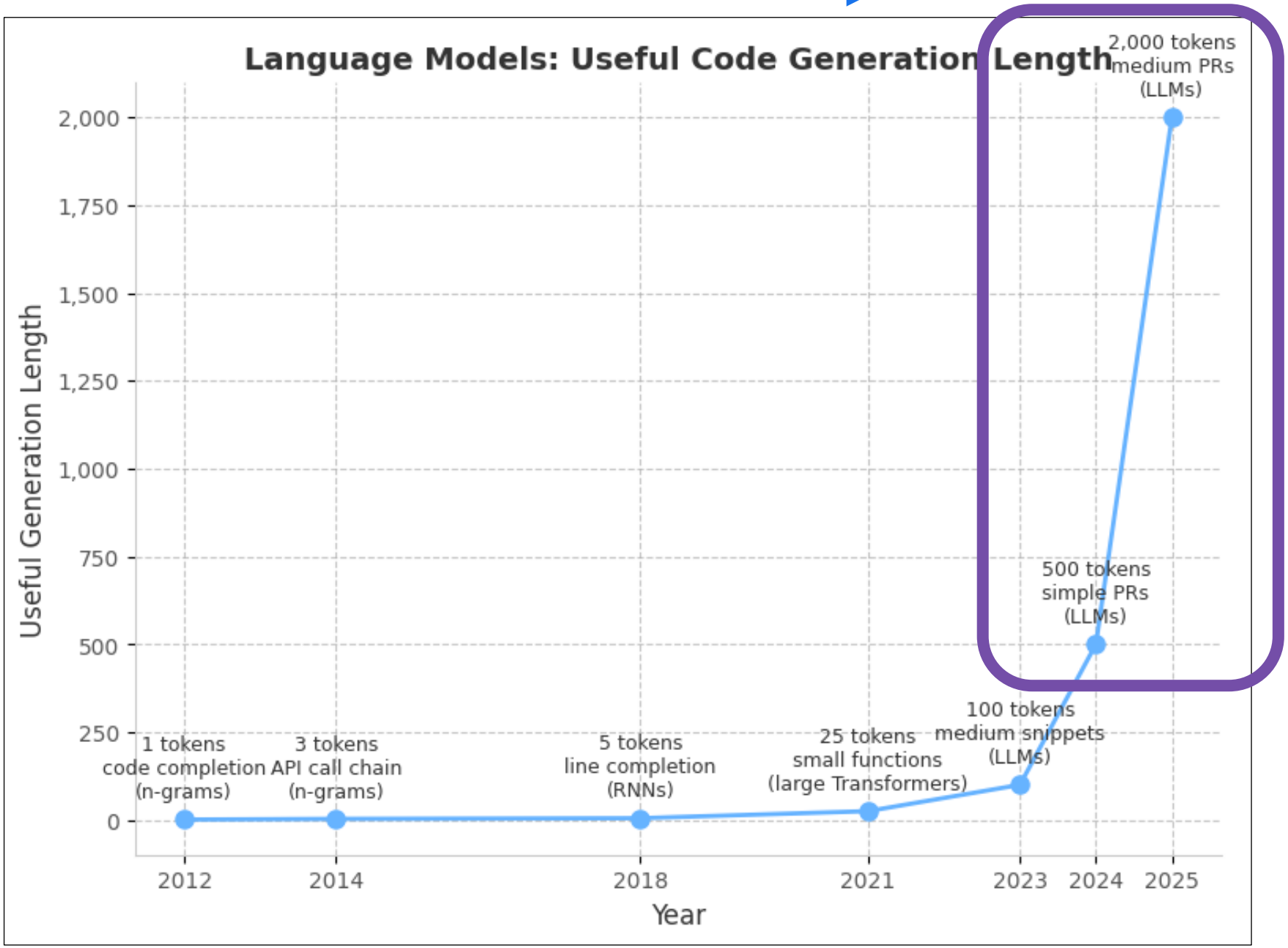
https://github.com/timsutton/brigadier/issues/52

Remember

Information out ≤ information in

'information in' = prompt + training data

past present future



Mostly agents

A generic SWE-Bench task

https://arxiv.org/pdf/2407.01489

Wait, why not just write the patch?

Because language models are:

Context-sensitive

Repositories are very large; code behavior is hard to predict

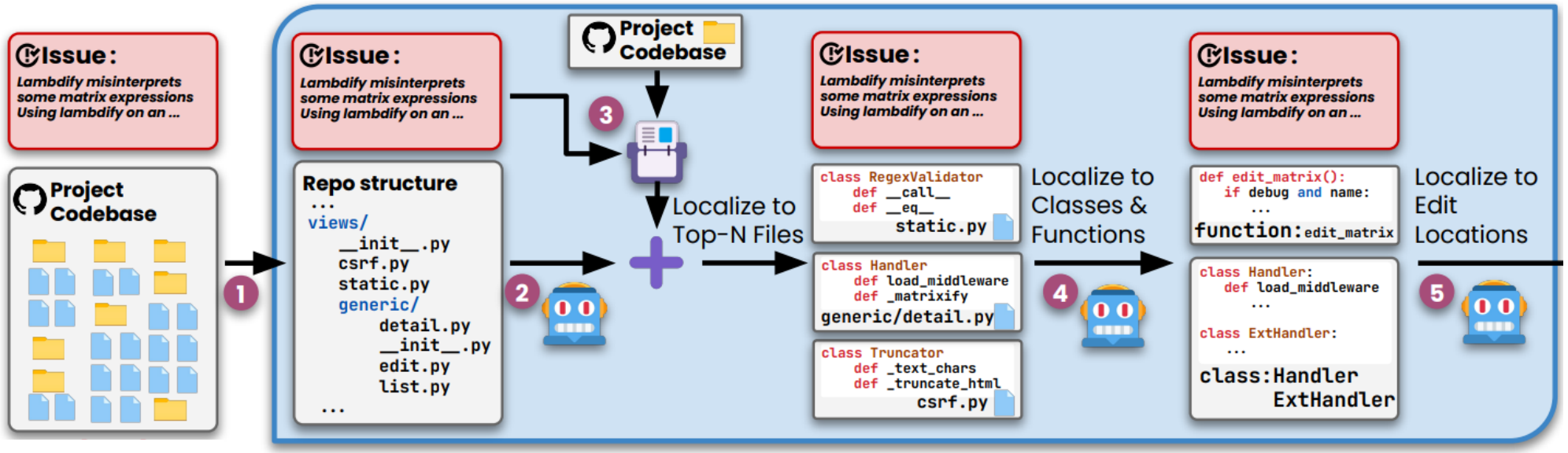
Approximate

Patches may be incorrect, may not solve the root cause

Distribution-specific

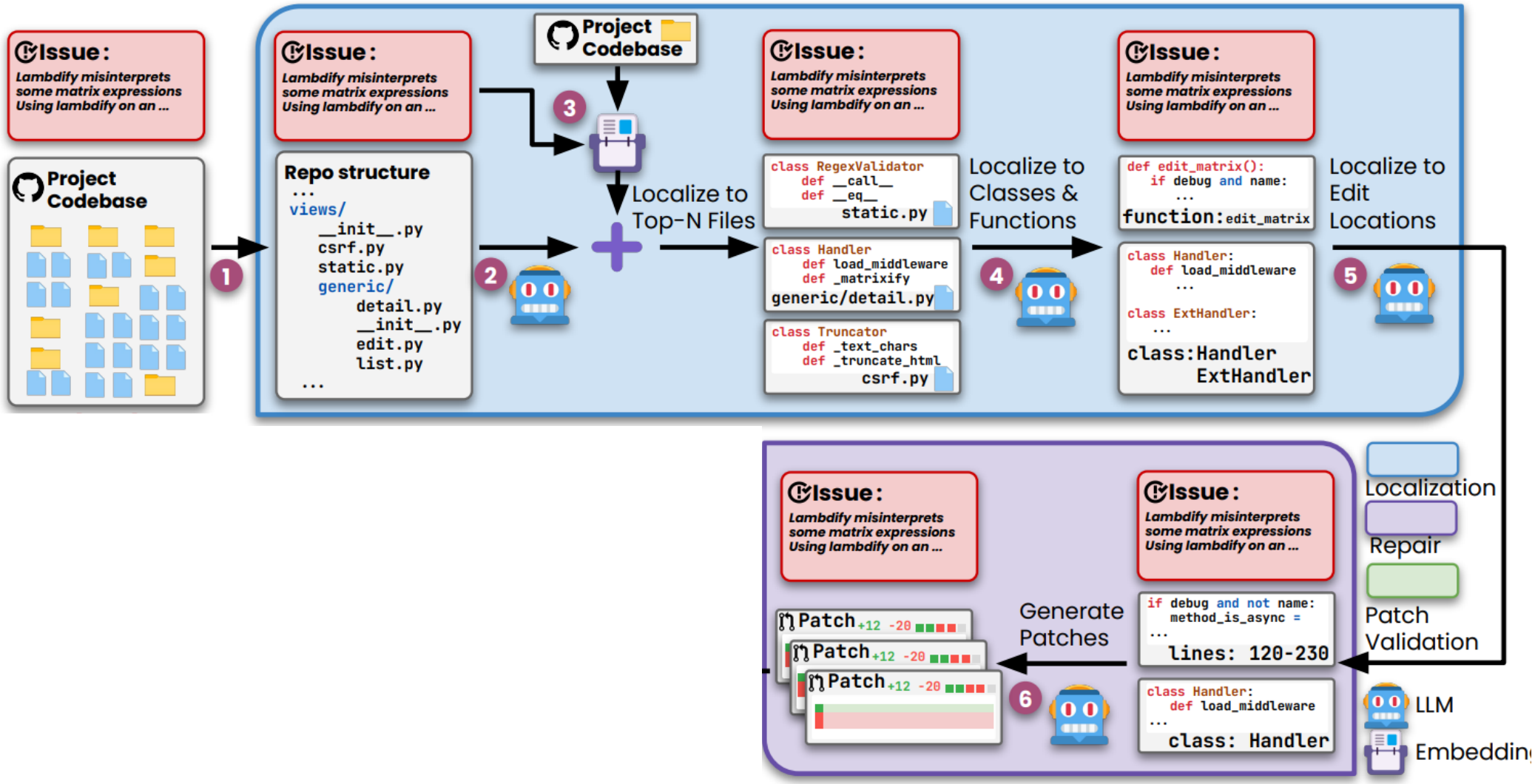
Requirements may not be complete, or ambiguous

Let's start *agentless*



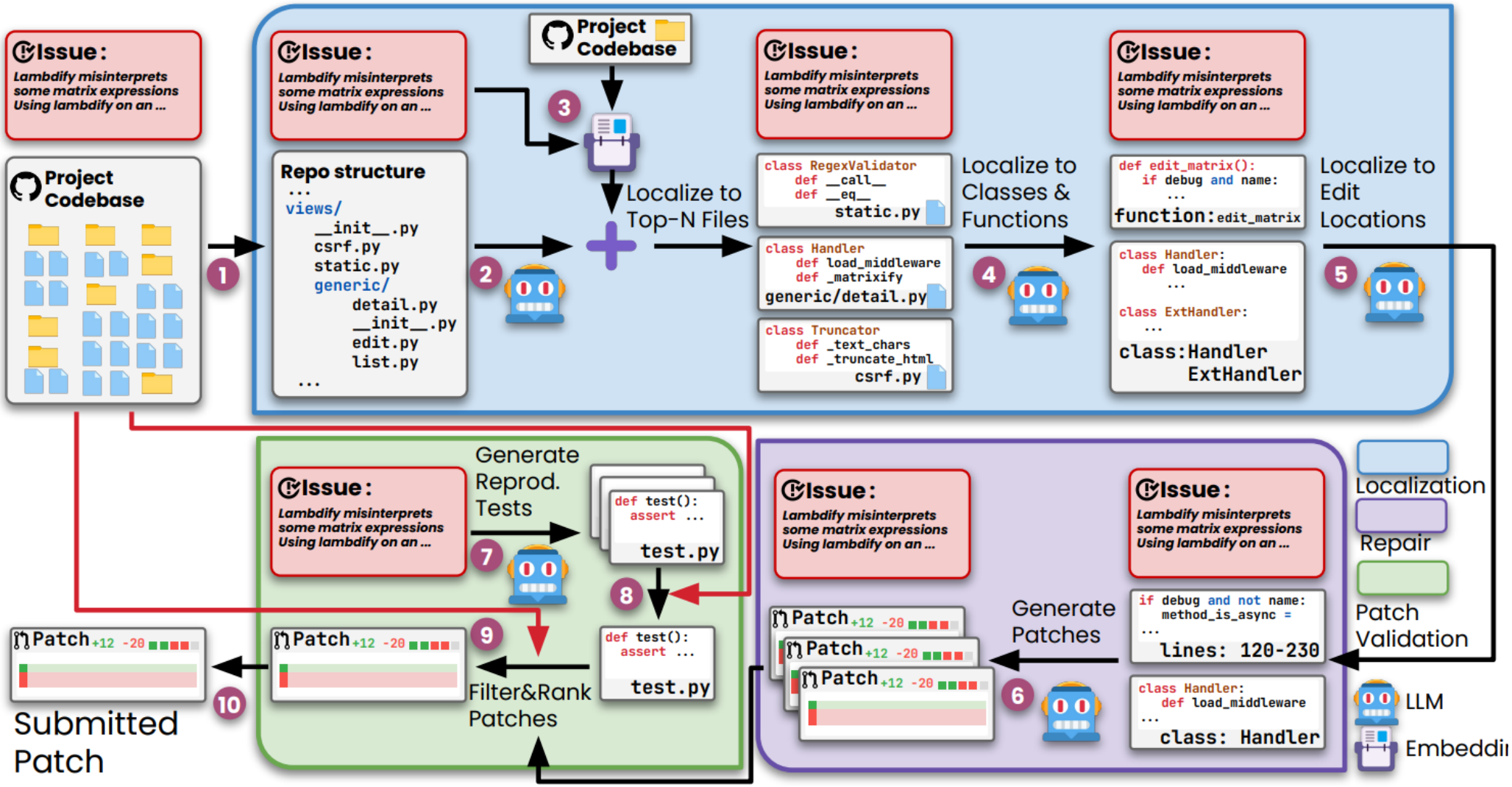
https://arxiv.org/pdf/2407.01489

Let's start *agentless*



https://arxiv.org/pdf/2407.01489

Let's start *agentless*

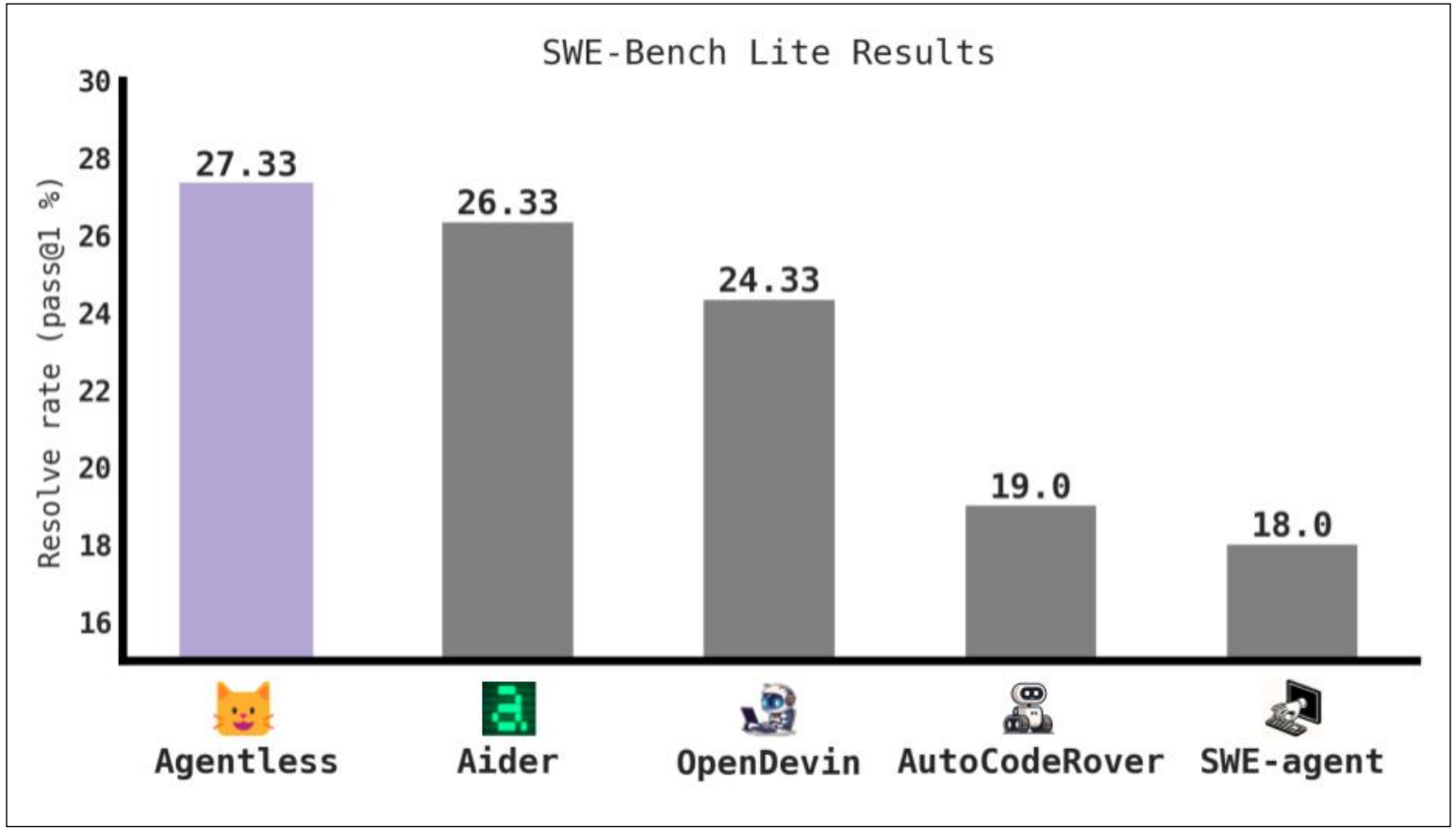


https://arxiv.org/pdf/2407.01489

This is perfectly decent!

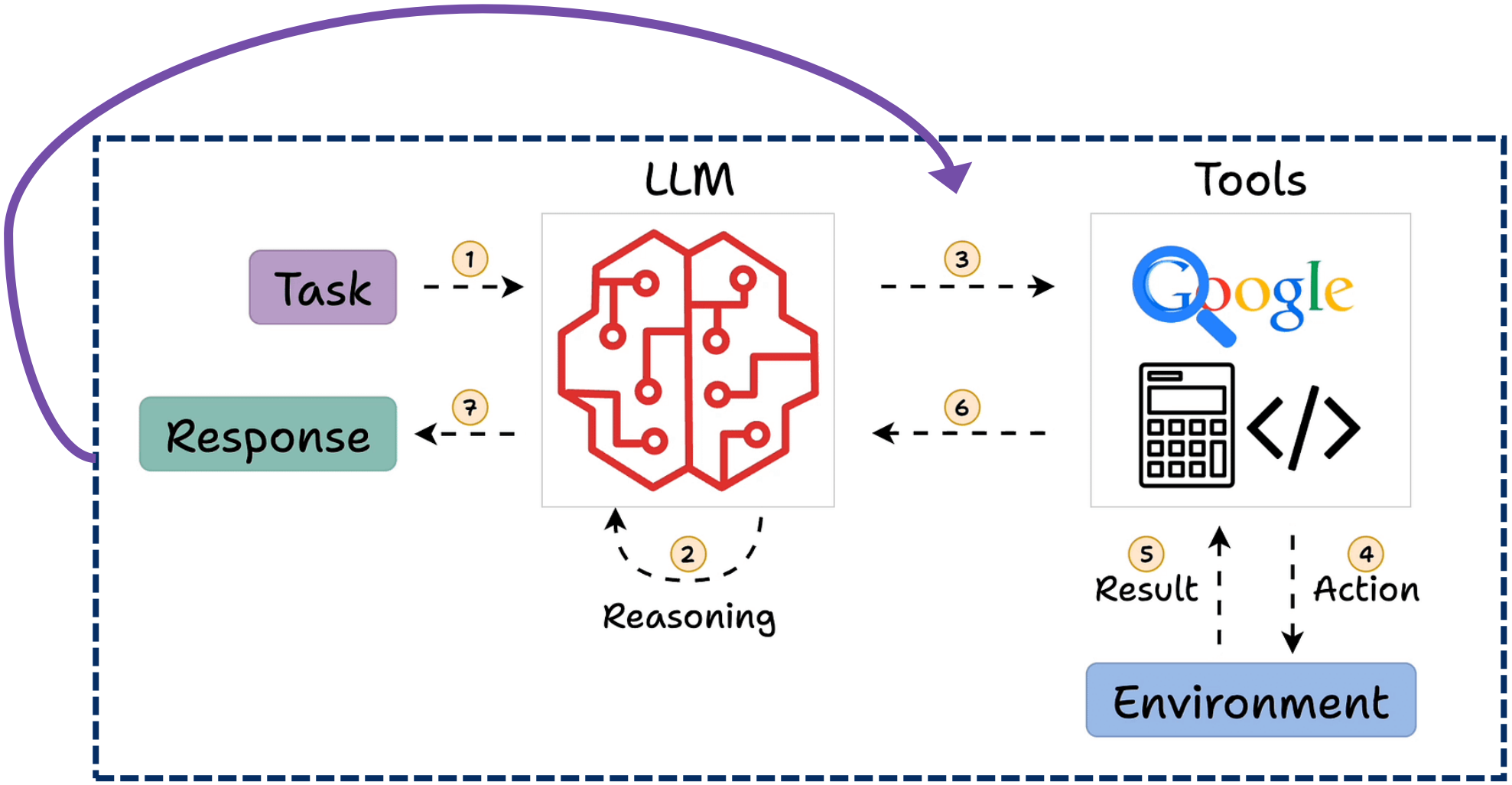
Cheap, better than many self-steering agents

For a while



https://arxiv.org/pdf/2407.01489

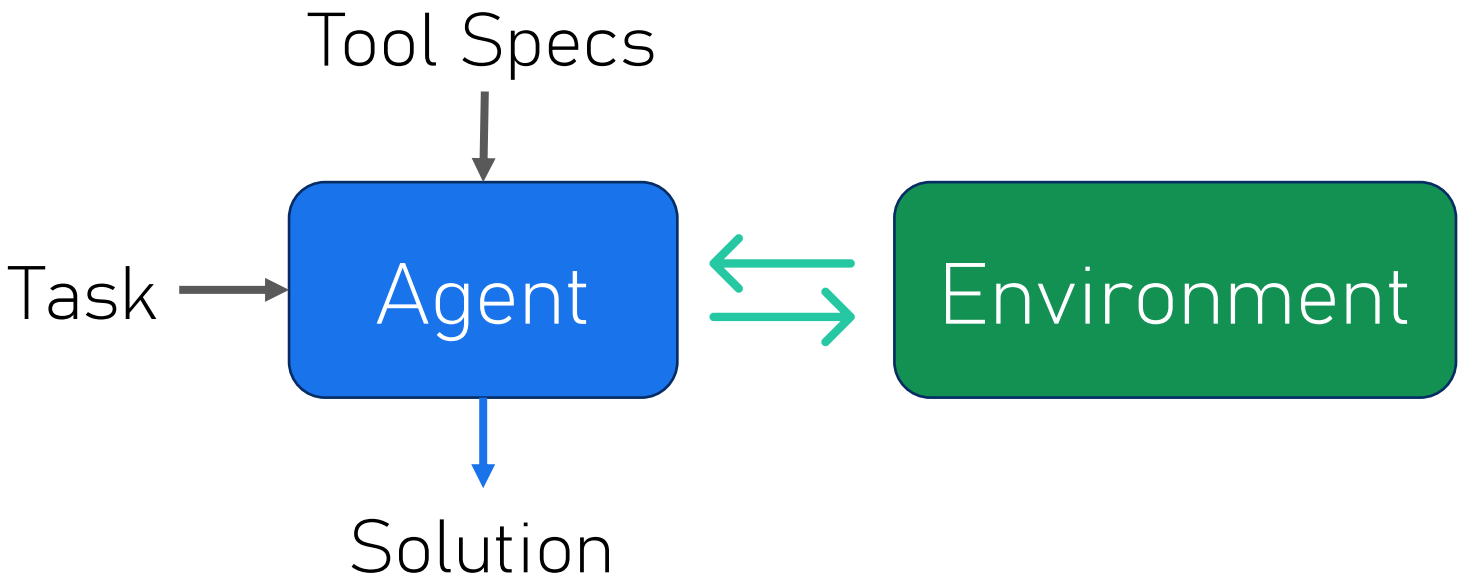
An agentic subroutine



https://blog.dailydoseofds.com/p/intro-to-react-reasoning-and-action

Agent Design

- Task + specs → thoughts + tool call
- Tool result → thoughts + (tool call | solution)



Agent Implementation: prompt + environment

Code Blame 72 lines (62 loc) · 5.19 KB Raw

```
You are OpenHands agent, a helpful AI assistant that can interact with a computer to solve tasks.

<ROLE>
Your primary role is to assist users by executing commands, modifying code, and solving technical problems effectively. You should be thorough, methodical, and prior
* If the user asks a question, like "why is X happening", don't try to fix the problem. Just give an answer to the question.
</ROLE>

<EFFICIENCY>
* Each action you take is somewhat expensive. Wherever possible, combine multiple actions into a single action, e.g. combine multiple bash commands into one, using s
* When exploring the codebase, use efficient tools like find, grep, and git commands with appropriate filters to minimize unnecessary operations.
</EFFICIENCY>

<FILE_SYSTEM_GUIDELINES>
* When a user provides a file path, do NOT assume it's relative to the current working directory. First explore the file system to locate the file before working on
* If asked to edit a file, edit the file directly, rather than creating a new file with a different filename.
* For global search-and-replace operations, consider using `sed` instead of opening file editors multiple times.
</FILE_SYSTEM_GUIDELINES>
```

https://github.com/All-Hands-AI/OpenHands/blob/main/openhands/agenthub/codeact_agent/prompts/system_prompt.j2

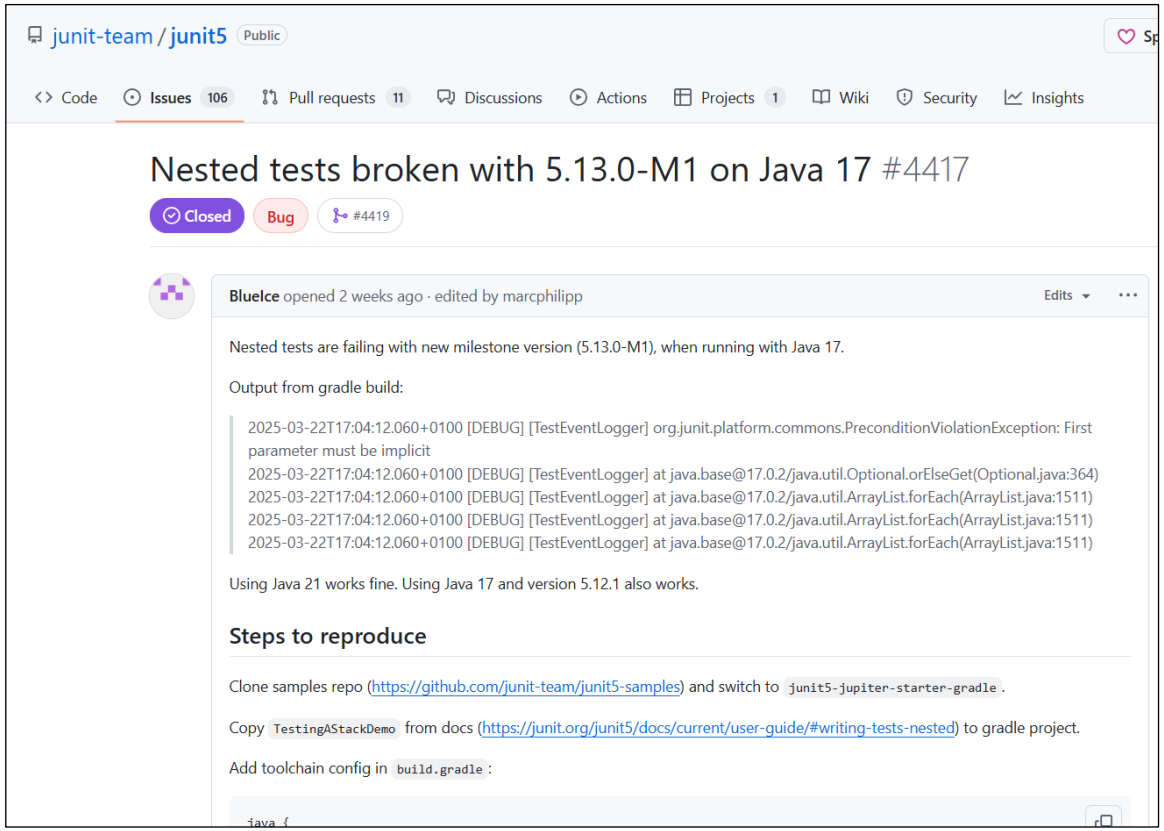
Agent Improvement: prompt tuning

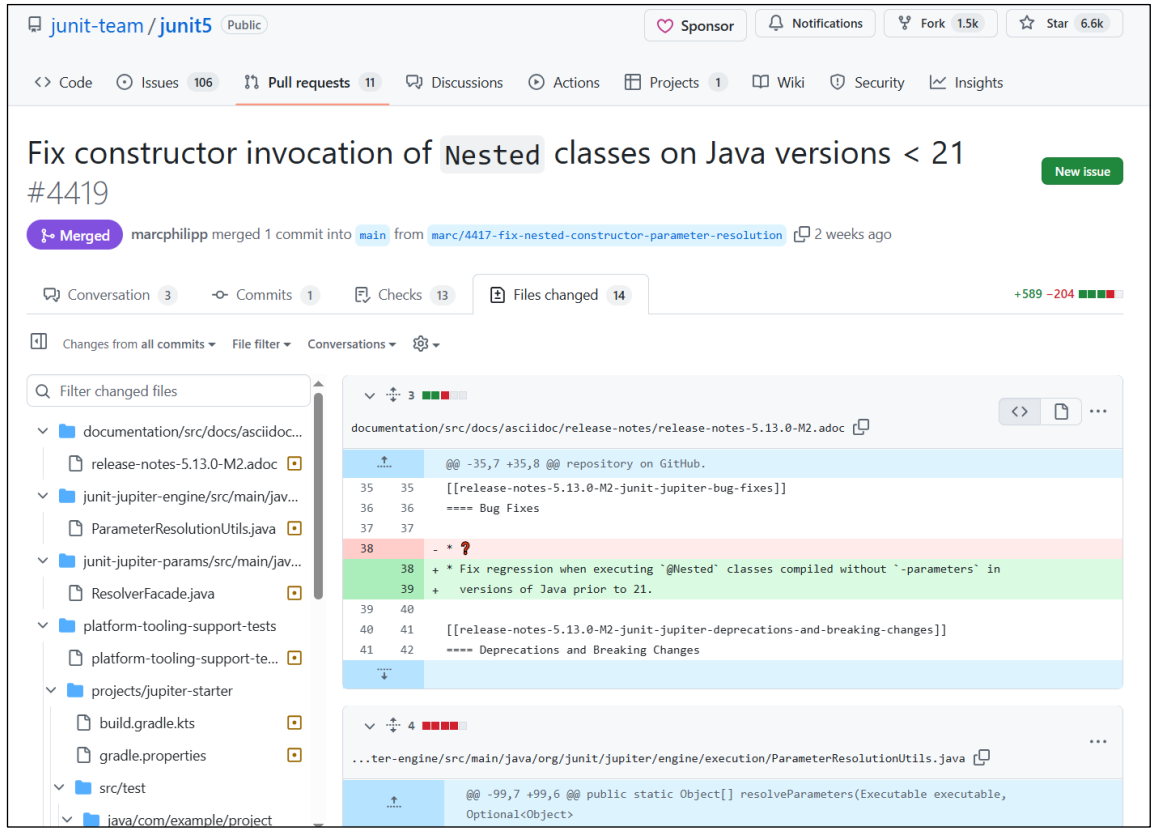
Code Blame 72 lines (62 loc) · 5.19 KB Raw

```
You are OpenHands agent, a helpful AI assistant that can interact with a computer to solve tasks.

<ROLE>
Your primary role is to assist users by executing commands, modifying code, and solving technical problems effectively. You should be thorough, methodical, and prior
* If the user asks a question, like "why is X happening", don't try to fix the problem. Just give an answer to the question.
</ROLE>

<EFFICIENCY>
* Each action you take is somewhat expensive. Wherever possible, combine multiple actions into a single action, e.g. combine multiple bash commands into one, using s
* When exploring the codebase, use efficient tools like find, grep, and git commands with appropriate filters to minimize unnecessary operations.
</EFFICIENCY>

<FILE_SYSTEM_GUIDELINES>
* When a user provides a file path, do NOT assume it's relative to the current working directory. First explore the file system to locate the file before working on
* If asked to edit a file, edit the file directly, rather than creating a new file with a different filename.
* For global search-and-replace operations, consider using `sed` instead of opening file editors multiple times.
</FILE_SYSTEM_GUIDELINES>

<CODE_QUALITY>
* Write clean, efficient code with minimal comments. Avoid redundancy in comments: Do not repeat information that can be easily inferred from the code itself.
* When implementing solutions, focus on making the minimal changes needed to solve the problem.
* Before implementing any changes, first thoroughly understand the codebase through exploration.
* If you are adding a lot of code to a function or file, consider splitting the function or file into smaller pieces when appropriate.
</CODE_QUALITY>
```

https://github.com/All-Hands-AI/OpenHands/blob/main/openhands/agenthub/codeact_agent/prompts/system_prompt.j2

Agent Improvement: fine-tuning

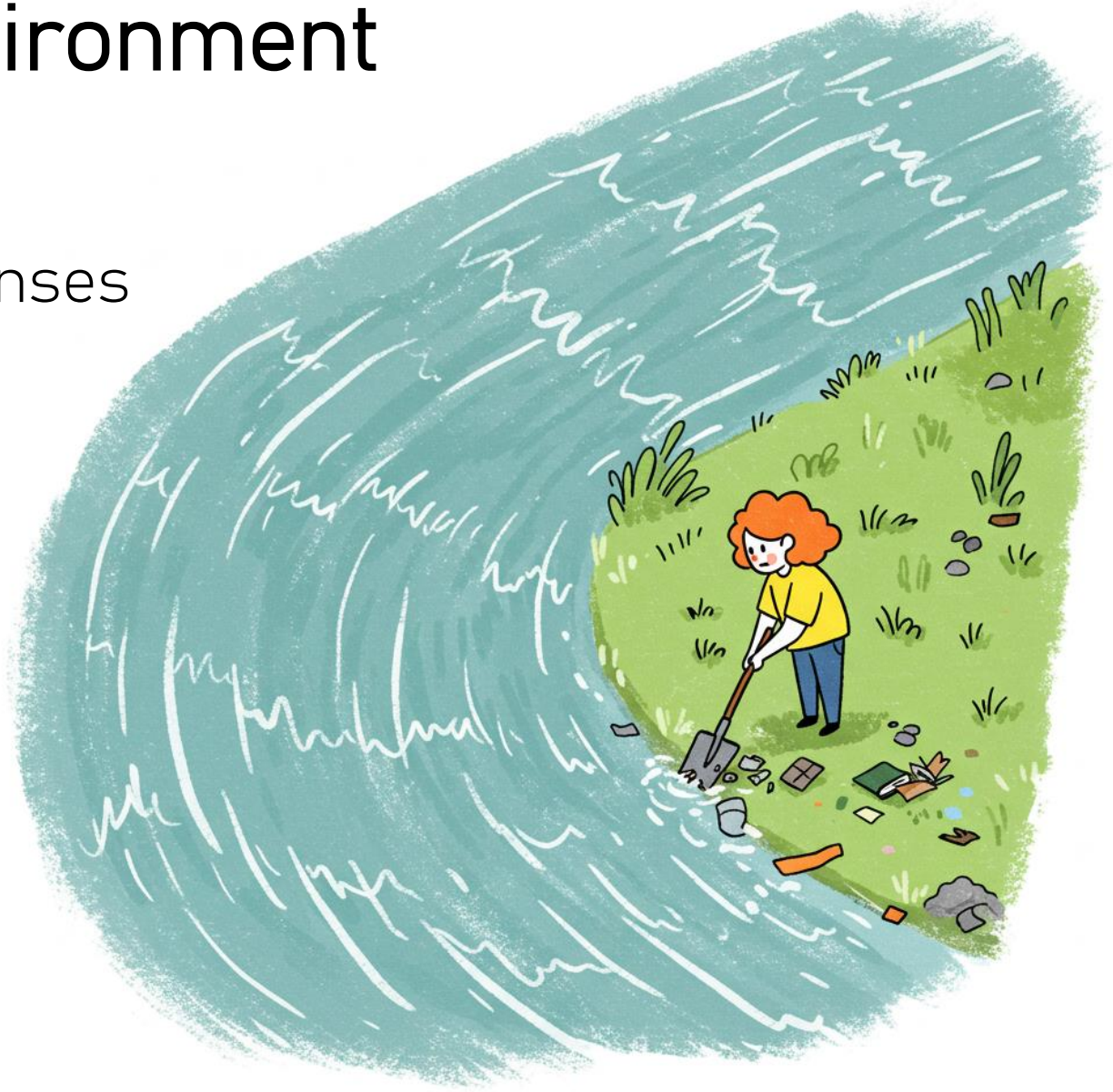




https://github.com/junit-team/junit5/issues/4417, https://github.com/junit-team/junit5/pull/4419

Agent Improvement: environment

- Reducing latency from tool responses
- Adding more tools
- Increasing robustness
- Parallelizing LLM & tool calls



Benchmarks: SWE-Bench

Model Input

▼ **Instructions** • 1 line

You will be provided with a partial code base and an issue statement explaining a problem to resolve.

▼ **Issue** • 67 lines

napoleon_use_param should also affect "other parameters" section Subject: napoleon_use_param should also affect "other parameters" section

Problem

Currently, napoleon always renders the Other parameters section as if napoleon_use_param was False, see source

```
def _parse_other_parameters_section(self, se...
    # type: (unicode) -> List[unicode]
    return self._format_fields(_('Other Para...

def _parse_parameters_section(self, section):
    # type: (unicode) -> List[unicode]
    fields = self._consume_fields()
    if self._config.napoleon_use_param: ...
```

▼ **Code** • 1431 lines

▶ **README.rst** • 132 lines

▶ **sphinx/ext/napoleon/docstring.py** • 1295 lines

▶ **Additional Instructions** • 57 lines

Gold Patch

sphinx/ext/napoleon/docstring.py

```
    def _parse_other_parameters_section(self, section: str) -> List[str]:
-       return self._format_fields(_('Other Parameters'), self._consume_fields())
+       if self._config.napoleon_use_param:
+           # Allow to declare multiple parameters at once (ex: x, y: int)
+           fields = self._consume_fields(multiple=True)
+           return self._format_docutils_params(fields)
+       else:
+           fields = self._consume_fields()
+           return self._format_fields(_('Other Parameters'), fields)
```

Generated Patch

sphinx/ext/napoleon/docstring.py

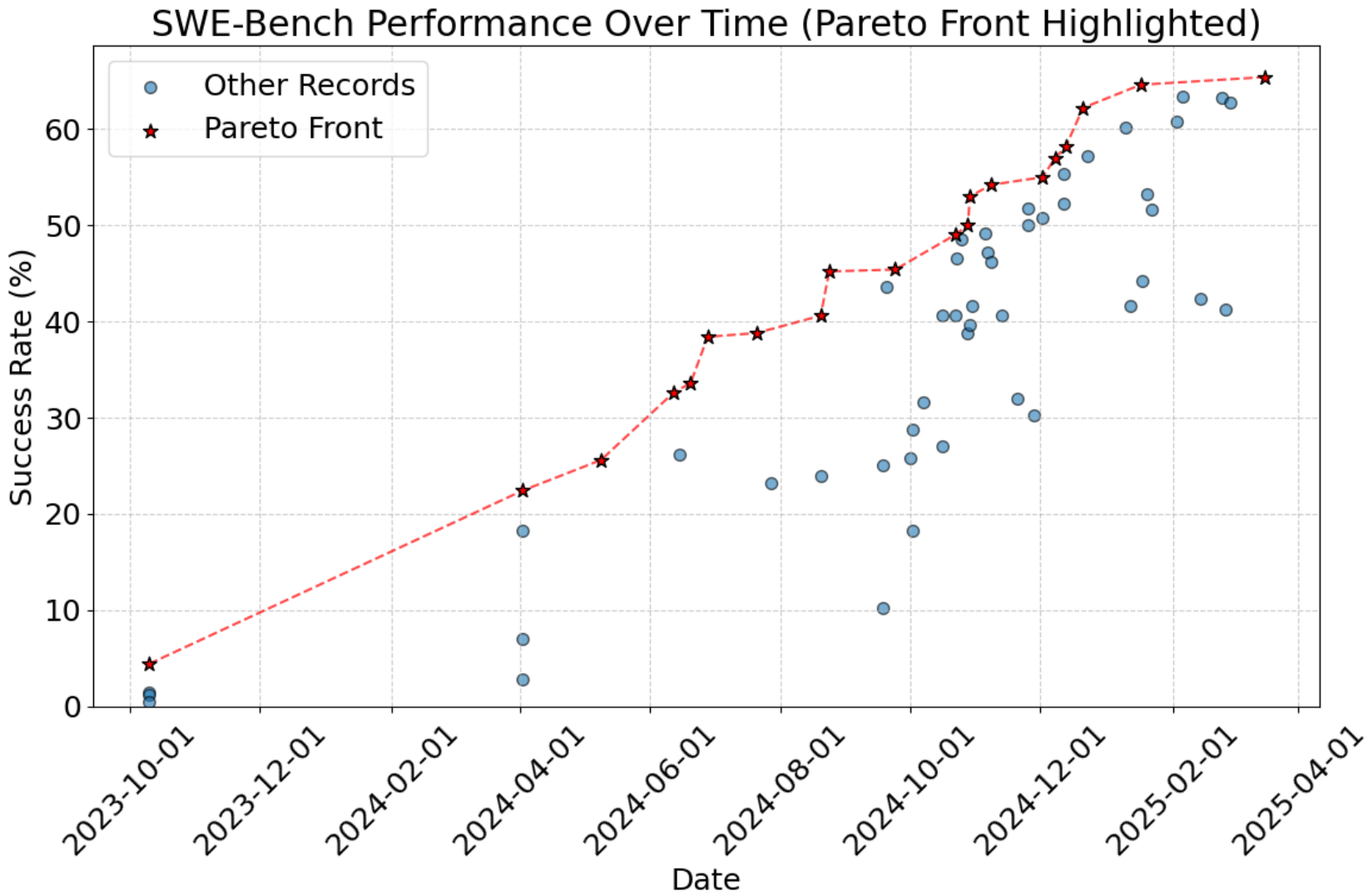
```
    def _parse_other_parameters_section(self, section: str) -> List[str]:
-       return self._format_fields(_('Other Parameters'), self._consume_fields())
+       return self._format_docutils_params(self._consume_fields())
```

Generated Patch Test Results

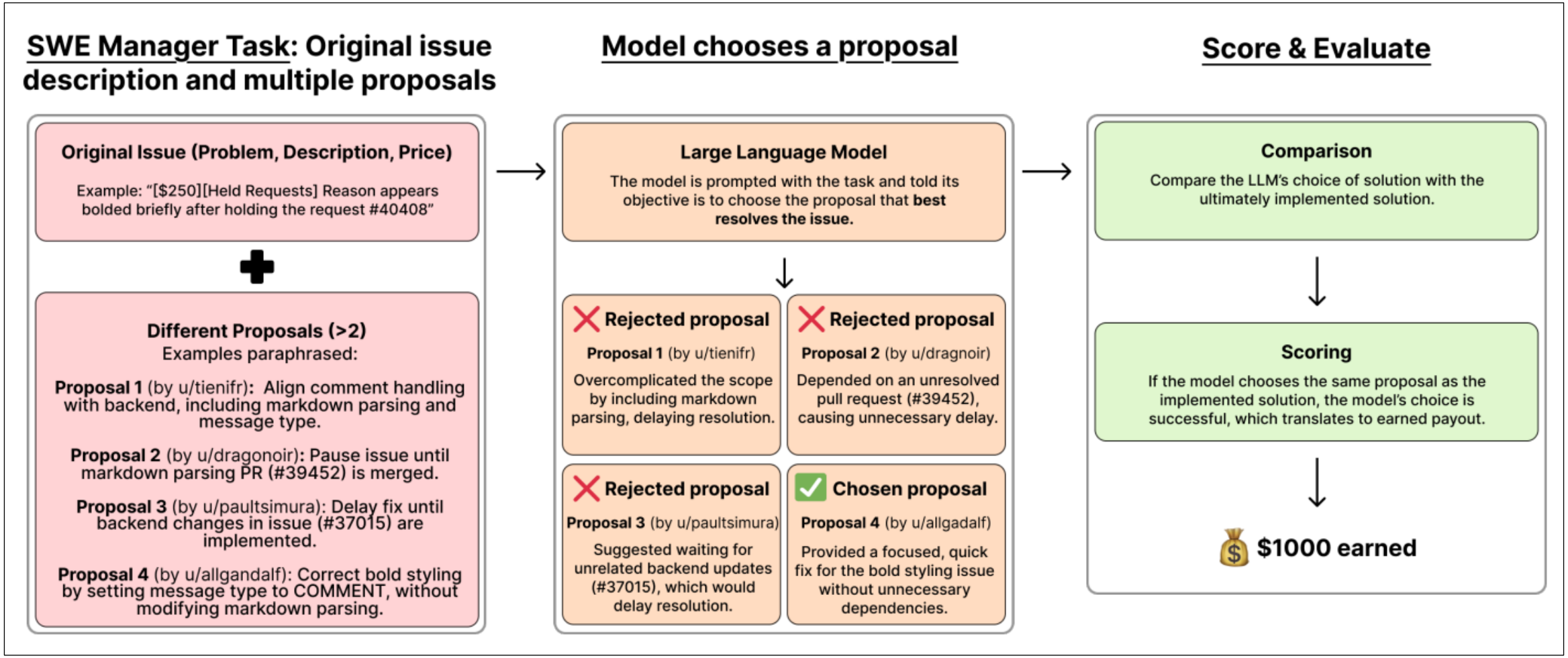
```
PASSED  NumpyDocstringTest (test_yield_types)
PASSED  TestNumpyDocstring (test_escape_args_and_kwargs 1)
PASSED  TestNumpyDocstring (test_escape_args_and_kwargs 2)
PASSED  TestNumpyDocstring (test_escape_args_and_kwargs 3)
PASSED  TestNumpyDocstring (test_pep526_annotations)
FAILED  NumpyDocstringTest (test_parameters_with_class_reference)
FAILED  TestNumpyDocstring (test_token_type_invalid)
===== 2 failed, 45 passed, 8 warnings in 5.16s =====
```

https://arxiv.org/pdf/2310.06770

Benchmarks: SWE-Bench



Benchmarks: SWE-Lancer



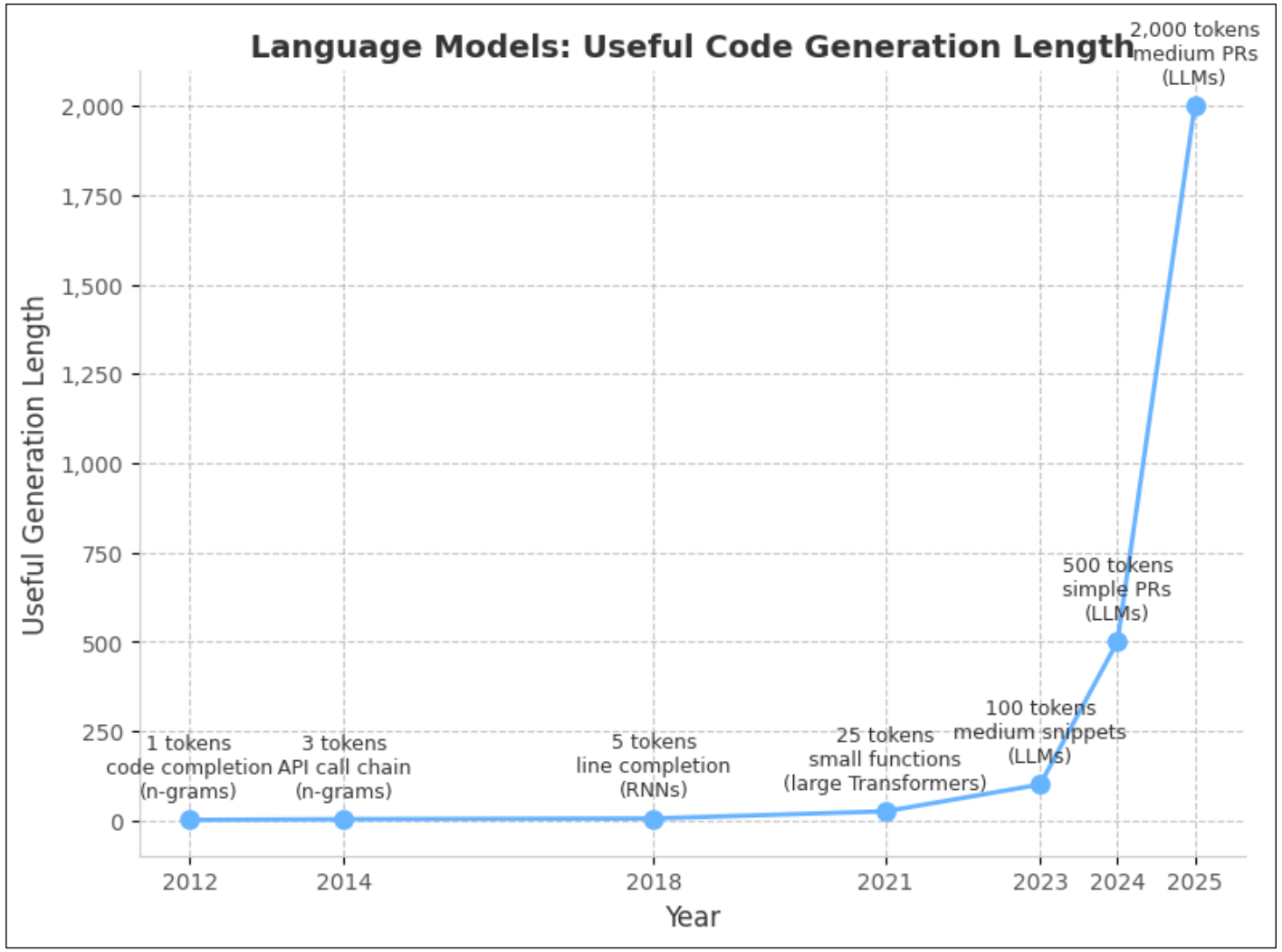
https://arxiv.org/abs/2502.12115

4x the tokens per year, forever?

Sort of, but no

Two concurrent effects:

- Models improving
- Software systems growing



Maybe 4x the tokens per year

For tasks that are very well-specified:

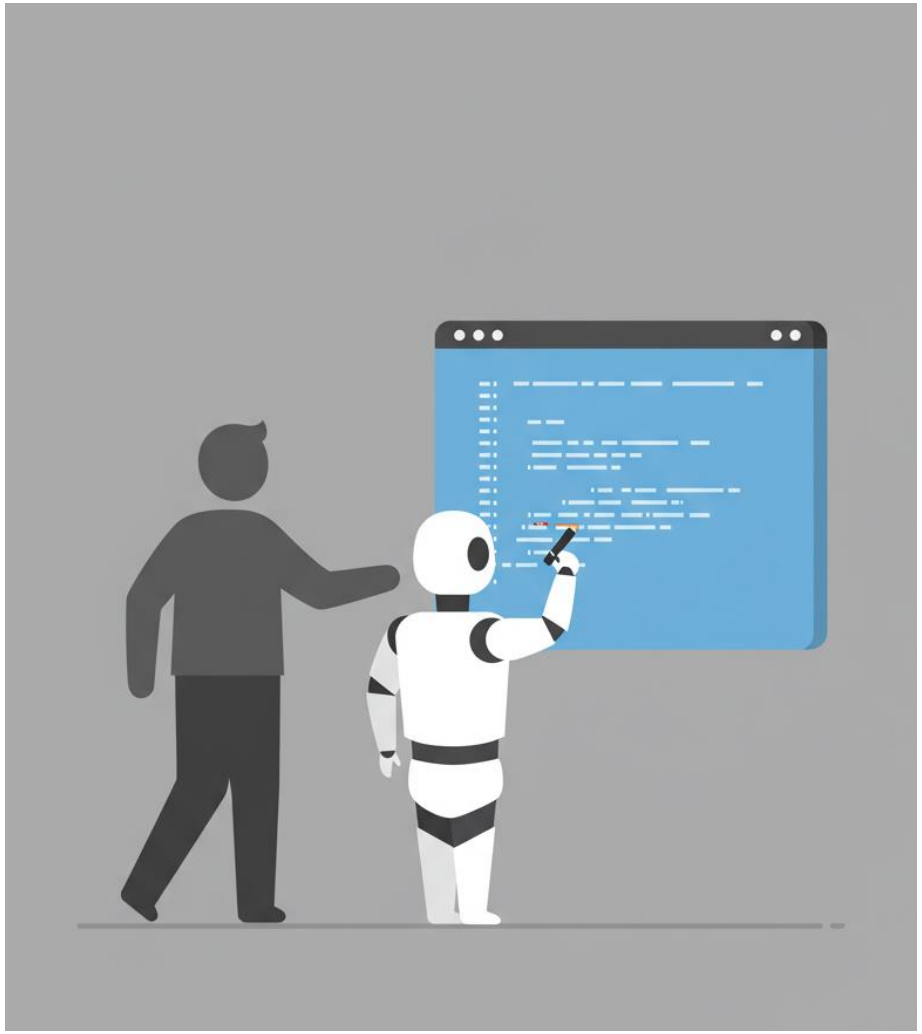
- 2025: Build a website with a donate button (~500 SLOC)
- 2026: Change the DB for a medium repo (2K SLOC)
- 2027: Add a new module based on detailed reqs (8K SLOC)
- 2028: Train a classifier and deploy to production (32K SLOC)
- 2029: Migrate a large project to a new language (128K SLOC)

Which means

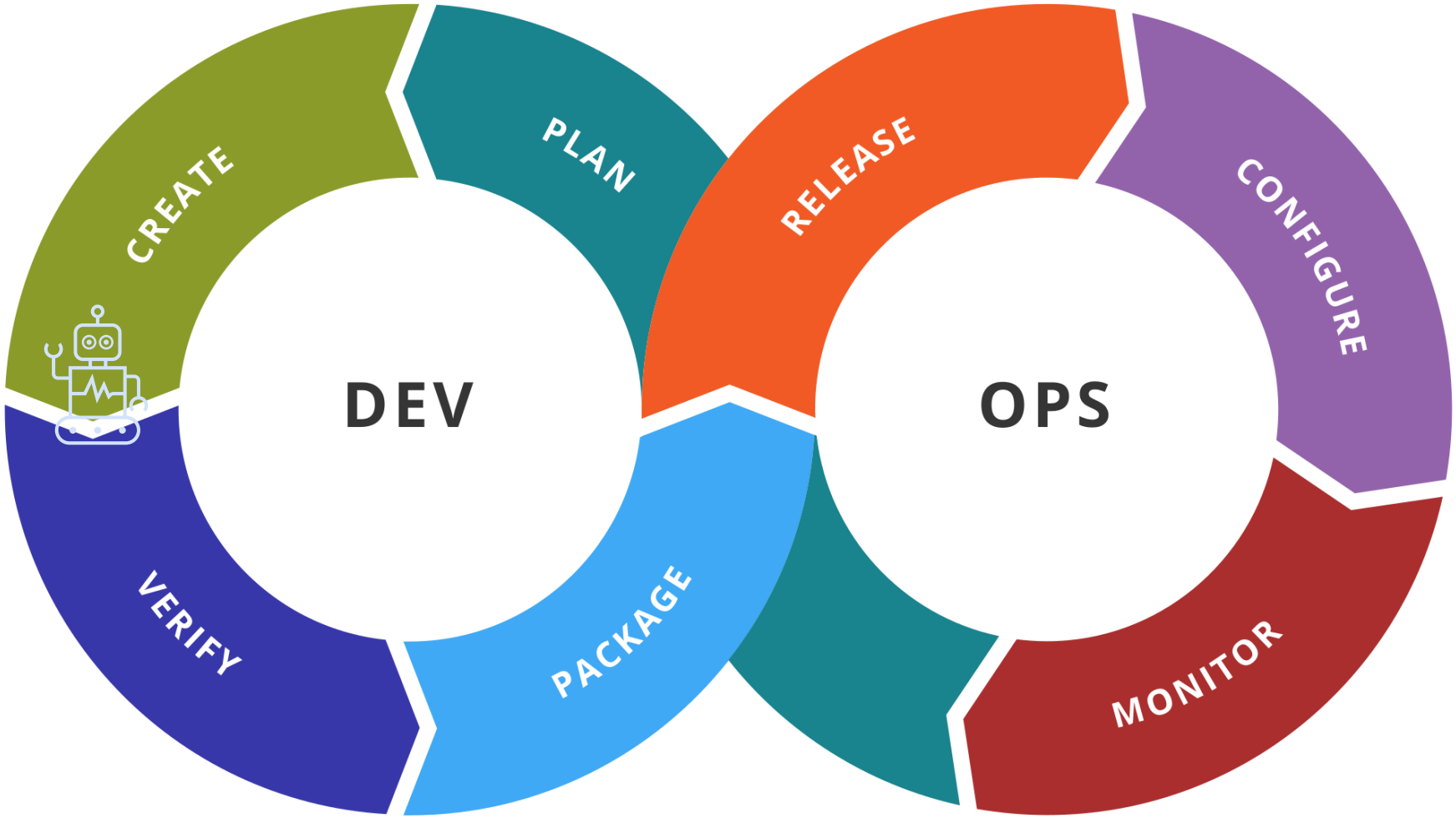
SWE Jobs are increasingly about:

specification and **verification**

- I'm writing increasingly detailed prompts as the models improve
- I regularly help an LLM debug

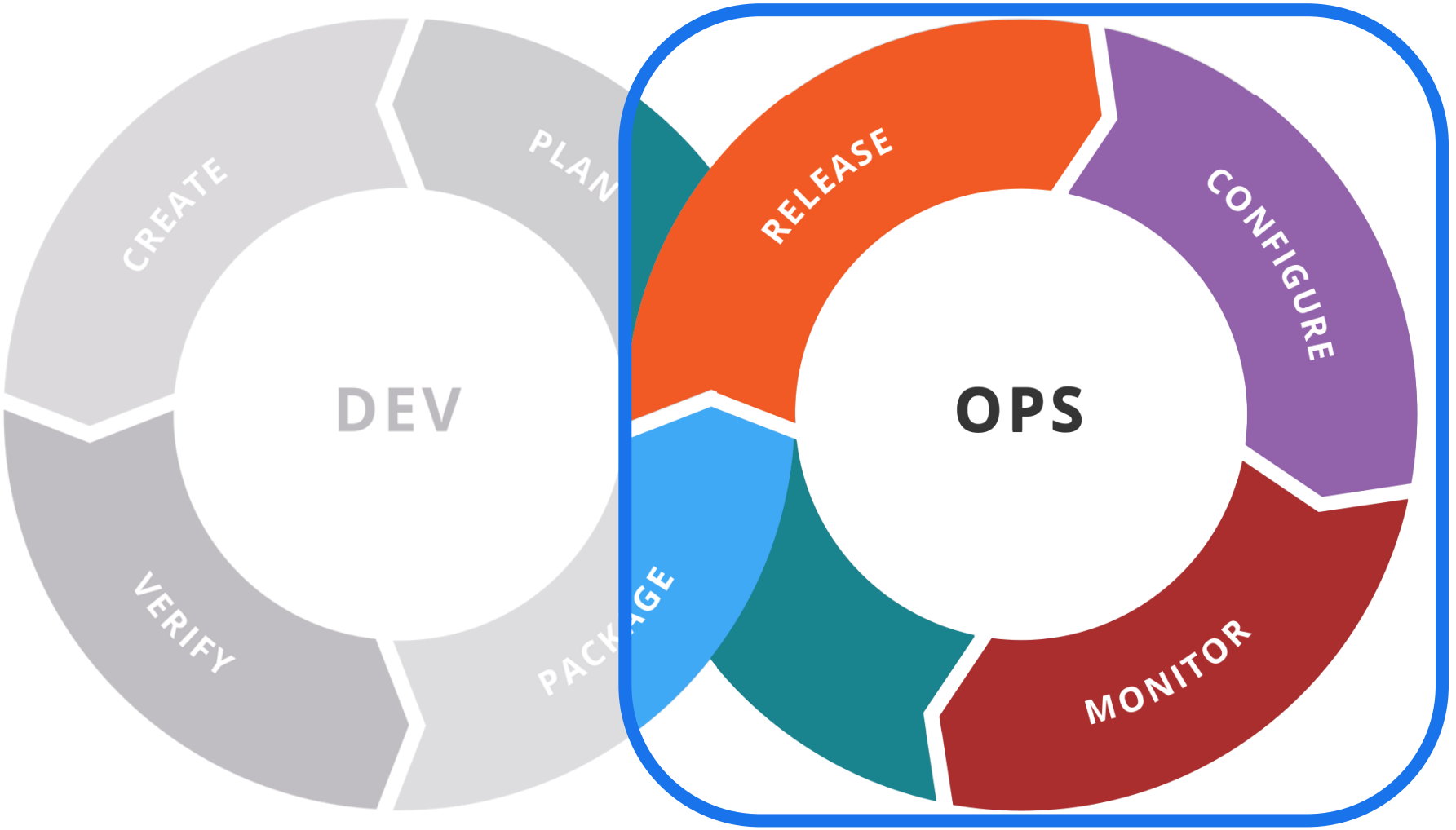


More generally, think tasks & artifacts



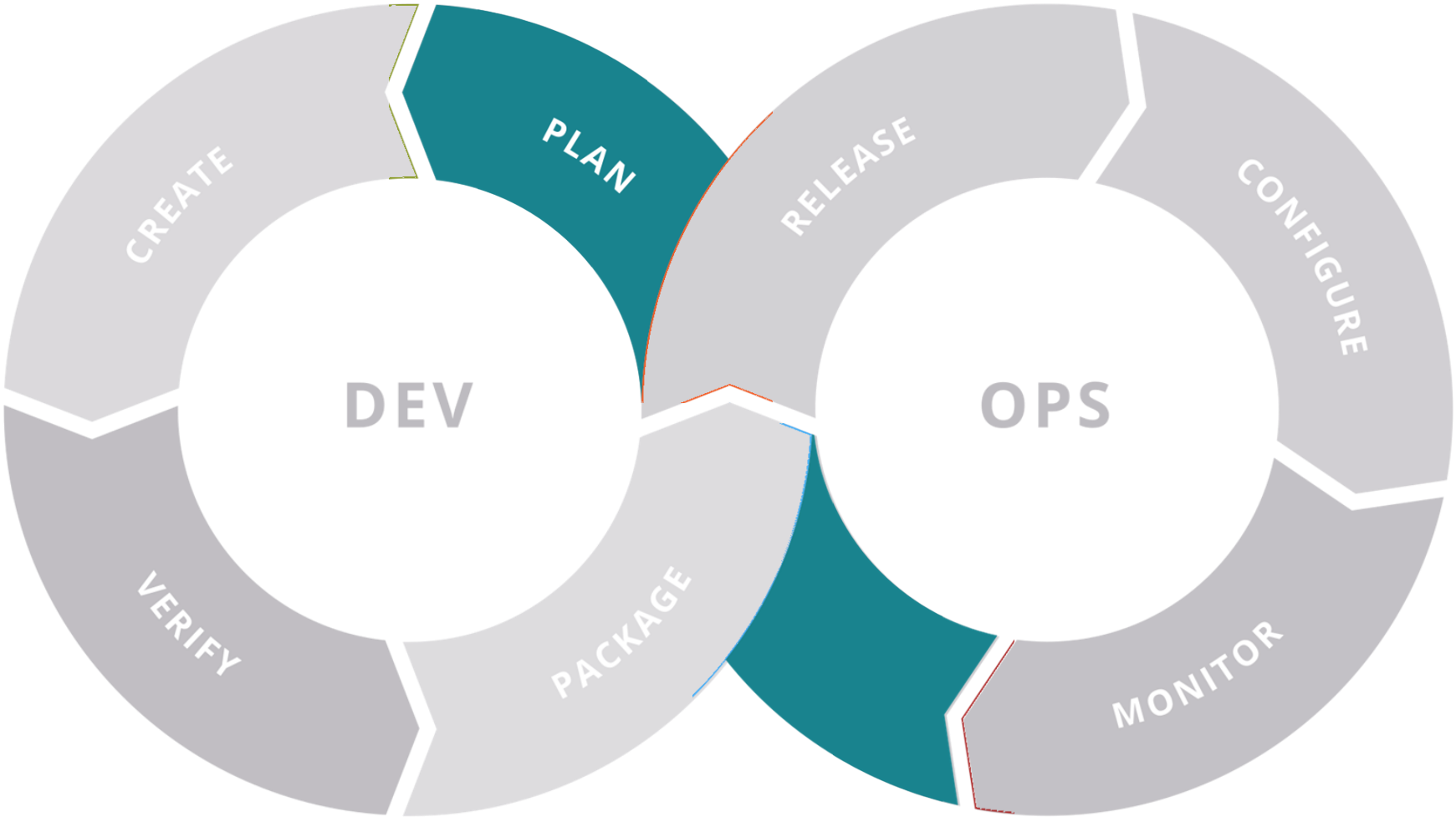
https://en.wikipedia.org/wiki/DevOps_toolchain#/media/File:Devops-toolchain.svg

Shifting Right



https://en.wikipedia.org/wiki/DevOps_toolchain#/media/File:Devops-toolchain.svg

Shifting Left

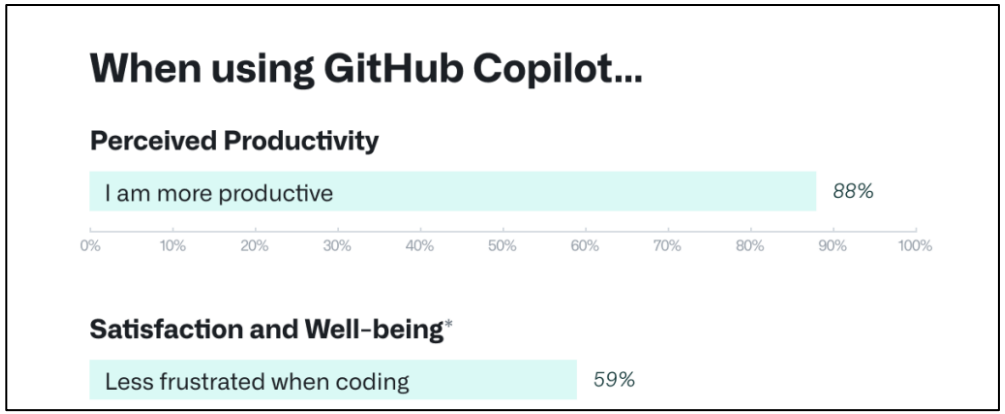


https://en.wikipedia.org/wiki/DevOps_toolchain#/media/File:Devops-toolchain.svg

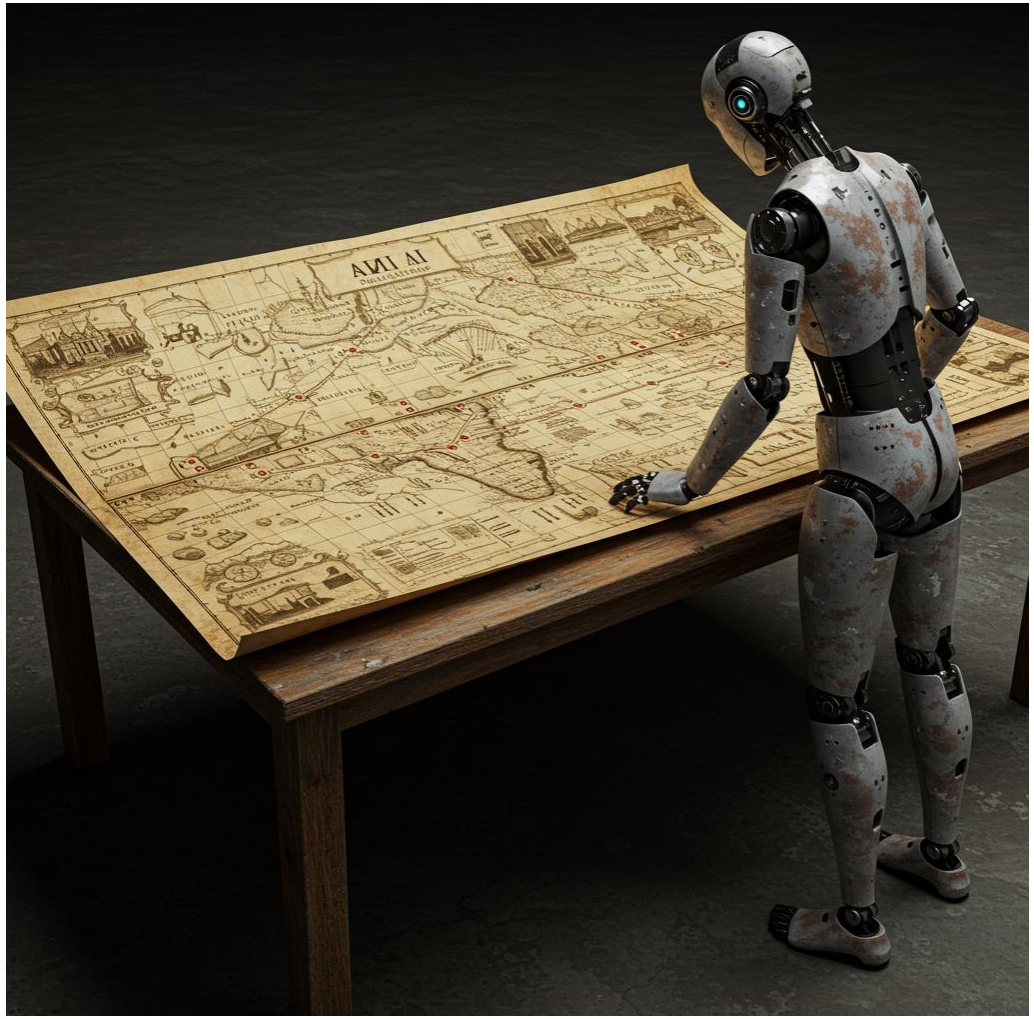
Advice for SE Professionals

Prepare for AI to do more and more of the coding

- Keep up with new tools
  - To help you calibrate
  - To speed up your work
- Prioritize people & design skills
  - AI doesn't help without knowledge of your users, system architecture
  - Information in ≤ information out



Agentic AI in Software Engineering



VHELLENDOORN@{CMU|GOOGLE}.COM

April 5th, 2025 Vincent Hellendoorn Google Deepmind & CMU